



College of Computing and Informatics

Computer Science Department

University of Sharjah

Spring 2025/2026

Optimization Tool for University Timetabling

*A project submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science*

By

Fatma I. Khammas (U23101181)

Fatima Z. Babakri (U23101777)

Maitha M. Alzarooni (U23101720)

Mirah M. Alsuwaidi (U23100915)

Supervised by

Dr. Saber M. Elsayed

Committee Member Names

Prof. Ahmed M. Khedr, Dr. Bouzaine Brik, Dr. Mian A. Jan

April 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Dr. Saber Elsayed for his guidance and support throughout this project. We also thank the Department of Computer Science in University of Sharjah for providing the necessary resources. Finally, we appreciate the efforts and collaboration of our team members in completing this project.

UNDERTAKING

This is to declare that the project entitled “Optimization Tool for University Timetabling” is an original work done by undersigned, in partial fulfillment of the requirements for the degree “Bachelor in Computer Science” at the Computer Science Department, College of Computing and Informatics, University of Sharjah, UAE.

All the analysis, design, and system development have been accomplished by the undersigned. Moreover, this project has not been submitted to any other college or university.



Fatma I.



Fatima Z.



Maitha M.



Mirah M.

ABSTRACT

This project presents a web-based university timetable optimization system designed to generate efficient and conflict-free schedules for courses, instructors, rooms, and timeslots. The motivation behind this work is the increasing complexity of manual timetable creation, which often leads to conflicts, inefficient resource usage, and difficulty in satisfying both institutional and student needs. The system addresses this problem by applying optimization techniques that consider both hard constraints, such as avoiding scheduling conflicts and respecting room capacities, and soft constraints, such as minimizing gaps and distributing courses effectively.

The proposed solution focuses on developing a flexible tool that supports multiple scheduling algorithms rather than relying on a single approach. This allows users to select from different algorithms and compare the generated timetables based on predefined evaluation criteria, enabling a more comprehensive analysis of solution quality and performance.

The expected outcome of this project is an automated tool that produces high-quality timetables while reducing manual effort and errors. This approach demonstrates that applying and comparing multiple optimization techniques can effectively handle complex scheduling problems and provides a foundation for future extensions, such as student-centered timetable customization based on individual preferences and constraints.

Contents

ACKNOWLEDGEMENT	ii
UNDERTAKING	iii
ABSTRACT	iv
List of Tables	viii
List of Figures	x
1 Introduction	1
1.1 Overview	1
1.2 Project Motivation	1
1.3 Problem Statement	1
1.4 Project Aim and Objectives	2
1.5 Project Scope	2
1.6 Project Software and Hardware Requirements	3
1.7 Project Limitations	3
1.8 Project Expected Output	3
1.9 Project Schedule	4
1.10 Project, Product, and Schedule Risks	4
1.11 Success Metrics and Acceptance Criteria	5
1.11.1 Success Metrics	5
1.11.2 Acceptance Criteria	5
1.12 Project Governance	6
1.12.1 Team Structure	6
1.12.2 Document and Version Control	6
2 Literature Review & Existing Systems	7
2.1 Introduction	7
2.1.1 Review of Optimization Approaches in Literature	7
2.2 Existing Systems	9
2.2.1 Overall Problems of Existing Systems	12
2.3 Overall Solution Approach	12
3 Problem Formulation	13

3.1	Sets	13
3.2	Decision Variables	14
3.3	Objective Function	14
3.4	Constraints	15
3.4.1	Hard Constraints	15
3.4.2	Soft Constraints	17
3.5	Boundary Conditions	17
3.6	Compact Problem Statement	17
4	Requirement Engineering and Analysis	19
4.1	Stakeholders	19
4.2	Use Case Diagram	20
4.2.1	Use Case Description	20
4.3	Non-Functional Requirements	26
4.4	Requirements Traceability Matrix	28
5	Architecture and Design	29
5.1	Overview	29
5.2	Software Architecture	29
5.2.1	Architecture Style	29
5.2.2	Architecture Components	29
5.3	Software Design	30
5.3.1	UML Sequence / Communication Diagrams	30
5.3.2	ER Diagram	38
5.4	User Interface Design (Prototype)	38
6	Methodology and System Implementation	43
6.1	System Overview and Design Rationale	43
6.2	Data Modeling and Representation	43
6.3	Algorithm Management Framework	43
6.4	Greedy Scheduling Algorithm	43
6.5	Genetic Algorithm for Timetabling	44
6.5.1	Solution Encoding	44
6.5.2	Initialization	44
6.5.3	Genetic Operators	44
6.5.4	Hyperparameters	45
6.5.5	GA Workflow	45
6.6	Constraint Management	46
6.7	Evaluation and Fitness Function	46
6.8	Database Integration	47
6.9	Implementation Technologies	47
7	Verification, Validation, and Evaluation	48

7.1	Evaluation Methodology	48
7.2	Test Strategy	48
7.2.1	Unit Testing	48
7.2.2	Integration Testing	49
7.2.3	System Testing	50
7.2.4	User Acceptance Testing	50
7.3	Optimization Evaluation Metrics	51
7.4	Experimental Results Overview	51
7.5	Security and Privacy Considerations	52
7.6	Functional Requirement Verification	53
7.7	Software Quality Assurance	54
8	Results and Discussion	55
8.1	Comparison and Evaluation	56
8.2	Genetic Algorithm Stability	56
8.3	Discussion	57
8.4	Limitations	57
9	Conclusion and Future Direction	58
9.1	Conclusion	58
9.2	Future Work	58
	Bibliography	60

List of Tables

1.1	Project Milestones and Tasks	4
1.2	Success Metrics	5
1.3	Team Roles and Responsibilities	6
2.1	Summary of Existing Tools	11
3.1	Sets	13
3.2	Parameters	14
3.3	General Hard Constraints	15
3.4	Hard Constraints for Dataset	16
3.5	Soft Constraints	17
4.1	Use Case of Login	21
4.2	Login Alternative Flows	21
4.3	Use Case of Manage Scheduling Data	21
4.4	Manage Scheduling Data Alternative Flows	22
4.5	Use Case of Generate Schedule	22
4.6	Generate Timetable Alternative Flows	23
4.7	Use Case of Compare Algorithms	23
4.8	Compare Algorithms Alternative Flows	24
4.9	Use Case of View Schedule	24
4.10	View Timetable Alternative Flows	24
4.11	Use Case of Set Preference	25
4.12	Set Preference Alternative Flows	25
4.13	Use Case of View Personal Schedule	25
4.14	View Personal Schedule Alternative Flows	25
4.15	Use Case of Create Personal Schedule	26
4.16	Create Personal Timetable Alternative Flows	26
4.17	List of Functional Requirements	28
6.1	GA Hyperparameters	45
7.1	Unit Test Cases – Fitness Function	49
7.2	Integration Test Cases	50
7.3	System Test Cases	50
7.4	User Acceptance Test Cases	50

7.5	Identified Risks and Mitigation Strategies	52
7.6	Requirements Test Cases	53
7.7	Software Quality Assessment	54
8.1	Conflict Breakdown (Base Schedule)	55
8.2	Conflict Breakdown (Genetic Algorithm)	55
8.3	Constraint Satisfaction Metrics	56
8.4	Computational Efficiency	56
8.5	Solution Quality Score	56

List of Figures

4.1	Use Case Diagram	20
5.1	Layered Architecture Diagram	29
5.2	Login Sequence Diagram	30
5.3	Manage Scheduling Data Sequence Diagram	31
5.4	Generate Schedule Sequence Diagram	32
5.5	Compare Algorithms Sequence Diagram	33
5.6	View Schedules Sequence Diagram	34
5.7	Set Preferences Sequence Diagram	35
5.8	View Personal Schedule Sequence Diagram	36
5.9	Create Personal Schedule Sequence Diagram	37
5.10	ER Diagram of the Database	38
5.11	Admin UI Page	39
5.12	Manage Data UI Page	40
5.13	Generate Schedule UI Page	41
5.14	View Schedules UI Page	41
5.15	Compare UI Page	42

CHAPTER 1: Introduction

1.1 Overview

This project focuses on the development of a web-based university timetable optimization system. The system aims to simplify the process of creating academic schedules by automatically assigning courses, instructors, rooms, and timeslots based on defined constraints. Timetabling in universities is a complex task due to limited resources and multiple scheduling requirements, which often makes manual scheduling inefficient and prone to conflicts.

The proposed system addresses this problem by generating optimized timetables that satisfy both hard constraints, such as avoiding scheduling conflicts and respecting room capacities, and soft constraints, such as improving time distribution and resource utilization. The project provides an automated solution that improves scheduling efficiency and reduces the effort required by administrators.

1.2 Project Motivation

The main motivation for developing this project is the difficulty of creating university timetables manually. Traditional scheduling methods often lead to conflicts between courses, inefficient use of rooms, and challenges in managing large numbers of students and instructors. These issues make the timetabling process time-consuming and likely to contain errors, which motivated the development of an automated solution.

This project is important because an efficient timetable system is essential for the smooth operation of any educational institution and can save time and resources. A well-structured timetable ensures that resources, such as rooms and instructors, are used effectively, while also reducing scheduling conflicts and improving overall academic organization.

The idea proposed in this project is the use of an optimization-based system that automatically generates timetables by considering both hard and soft constraints. Unlike manual methods, the system applies intelligent techniques to produce better-quality schedules that are balanced, conflict-free, and more efficient for university use.

1.3 Problem Statement

University timetabling is a combinatorial optimization problem that requires assigning a set of course sections to rooms and timeslots while satisfying a defined set of constraints. The problem is classified as NP-hard, meaning that the number of feasible combinations grows exponentially as the number of sections, rooms, and timeslots increases, making exhaustive search computationally infeasible at a real-world scale.

At the University of Sharjah, the scheduling process involves approximately 4,700 course sections, 375 rooms across multiple campuses, and 479 available timeslots per semester. The

current approach relies on a manually constructed base schedule that is built on historical scheduling practices and institutional knowledge accumulated over multiple semesters. While this base schedule achieves complete section coverage, it produces 4,112 constraint violations when evaluated against formalized scheduling rules, including instructor double-bookings, room capacity violations, room type mismatches, and campus assignment errors. This indicates that the manual process, while practically functional, does not systematically optimize against defined constraints and relies heavily on implicit knowledge that cannot be easily transferred, audited, or scaled.

The specific problem addressed in this project is the automated assignment of pre-defined course sections to valid combinations of rooms and timeslots, subject to hard constraints that must be strictly satisfied and soft constraints that should be minimized where possible.

1.4 Project Aim and Objectives

The aim of this project is to develop an easy-to-use web-based university timetable optimization system that can automatically generate efficient and conflict-free schedules for courses, instructors, rooms, and timeslots.

This goal is achieved by designing and implementing a system that uses input data such as course requirements, instructor availability, room capacities, and scheduling constraints. The system applies optimization techniques to ensure that all hard constraints are satisfied, such as avoiding time clashes and exceeding room limits, while also improving soft constraints like better distribution of courses and more efficient use of resources.

1.5 Project Scope

This phase of the project is targeted towards the registration department/administrations as the primary users. The system is implemented as a functional prototype that demonstrates the feasibility of applying automated optimization techniques to real-world academic scheduling.

The core scheduling entities addressed in this phase are sections, timeslots, and rooms. A section is defined as a course instance with an assigned instructor, representing the fundamental unit of scheduling. While instructor-centered features such as preference management and workload balancing are deferred to future development, instructor identity is incorporated into the current constraint model to prevent double-booking conflicts.

Two scheduling algorithms are implemented and evaluated: a greedy heuristic serving as a deterministic baseline and a genetic algorithm serving as the metaheuristic optimization approach. Both algorithms operate on the same dataset and are evaluated under a unified constraint validation and fitness evaluation framework, enabling direct performance comparison.

The system is developed as a prototype with two parallel components that are not yet fully integrated. The Python backend handles all optimization logic, database communication, and schedule generation, with results visualized through a temporary interface used for evaluation and demonstration. A frontend has been designed and implemented to represent the intended administrator-facing interface, including data management views, schedule generation controls,

and result displays. However, a live connection between the frontend and backend is not completed in this phase and represents the primary integration milestone for the next development stage. All experimental results reported in Chapter 8 are produced through the backend and temporary visualization layer.

The following capabilities are within scope for this phase: schedule generation satisfying hard constraints, constraint violation detection and reporting, algorithm performance comparison, and administrator-facing schedule visualization. The following are explicitly deferred to future phases: student-facing schedule creation, instructor preference integration, real-time database synchronization, and multi-campus deployment.

1.6 Project Software and Hardware Requirements

Software Requirements

- Programming language: Python
- GUI framework: Node.js, React
- Database: Supabase / PostgreSQL
- Development tools: Visual Studio Code
- Version control: GitHub
- Operating system: Windows / macOS

Hardware Requirements

- Processor: Minimum Intel i5 or equivalent
- RAM: Minimum 8 GB
- Storage: At least 500 MB free space

1.7 Project Limitations

The system is integrated with the university-provided database to retrieve and manage real academic data, ensuring that timetable generation is based on accurate and relevant information. It is designed to process this data and generate schedules according to defined hard and soft constraints.

However, the project is implemented as a prototype and does not include full institutional deployment or real-time synchronization with external systems. Advanced features such as personalized student timetables and frontend-to-backend integration are not included in this stage of development and are intended to be milestones for the senior project.

1.8 Project Expected Output

The desired outcome of this project is a functional web-based university timetable optimization system that can automatically generate feasible and conflict-free schedules for courses, instructors, rooms, and timeslots. The system is expected to correctly apply scheduling con-

straints to ensure that no time clashes occur and that room capacities are not exceeded.

In addition, the system should produce well-organized timetables that improve the distribution of classes across available time slots and make efficient use of university resources. By using optimization techniques and real academic data from the university database, the system is expected to reduce manual scheduling effort and provide consistent and reliable timetable generation results.

1.9 Project Schedule

Table 1.1: Project Milestones and Tasks

Milestone	Date	Key Tasks
Project Initialization	2 Feb – 5 Feb	Finalize team charter; tools setup
Requirements Gathering	9 Feb – 12 Feb	Gather functional and non-functional requirements; identify constraints and objectives
Literature Review	16 Feb – 19 Feb	Review existing timetabling approaches; research and compare optimization algorithms
Progress Report #1 & System Design	23 Feb – 26 Feb	Progress report 1 submission; design system architecture and data model
GUI Design	27 Feb – 2 Mar	Design web interface for the timetabling prototype
Progress Report #2 & Prototype Development	23 Mar – 13 Apr	Implement core optimization algorithm; implement frontend and backend components; create dashboard to compare algorithm results side by side; progress report 2 submission
Testing and Refinement	14 Apr – 16 Apr	Perform functional testing; refine user experience; ensure feasible schedules
Documentation	20 Apr – 23 Apr	Prepare final report
Final Presentation	27 Apr – 30 Apr	Project presentation

1.10 Project, Product, and Schedule Risks

One of the main risks in this project is the possibility of development taking longer than the planned schedule. This may occur due to unexpected technical difficulties, such as challenges in implementing the optimization algorithm, integrating the system with the university database, or handling complex scheduling constraints. Delays may also result from debugging issues, requirement changes, or limited experience with certain development tools and frameworks. To

reduce this risk, the project is planned in clear milestones with regular progress checks, allowing early detection of issues and timely adjustments to the schedule. This helps ensure that the project is completed as close as possible to the intended timeline.

1.11 Success Metrics and Acceptance Criteria

1.11.1 Success Metrics

Table 1.2: Success Metrics

Metric	Description	Target Value	How It Is Measured
Feasibility	No hard constraint violations	100% feasible solutions	Constraint validation check
Scheduling Rate	Percentage of sections successfully scheduled	$\geq 90\%$ total sections	—
Conflict Rate	Number of instructor/room conflicts	$< 10\%$	Constraint validation
Fitness Score	Overall quality of the timetable	Maximized	Fitness function $F(X)$
Execution Time	Time to generate a timetable	≤ 20 min	System timer

1.11.2 Acceptance Criteria

The project will be considered successfully completed and accepted when the following conditions are met:

- The system must successfully generate university timetables based on the given courses, instructors, rooms, and timeslots.
- The system must correctly handle scheduling constraints and avoid conflicts such as overlapping classes and room overbooking.
- The main system features, including data input, timetable generation, and result display, must function without critical errors.
- The system must be properly integrated with the provided university database and retrieve data accurately.
- All test cases must be executed and passed to ensure system reliability and correctness.

1.12 Project Governance

1.12.1 Team Structure

Table 1.3: Team Roles and Responsibilities

Team Member	Role	Responsibilities
Fatima Z. Babakri	Project Manager	Project planning and task tracking; team coordination and supervisor communication; version control and deployment support
Maitha M. Alzarooni	Solution Architect	Designs the overall system architecture; researches and selects optimization algorithms; defines constraints and optimization objectives
Mirah M. Alsuwaidi	Backend Developer	Implements optimization and scheduling logic; develops backend APIs; designs and manages the database
Fatma I. Khammas	Frontend Developer & QA Support	Builds the web interface; designs user workflows and UI; conducts functional testing and usability validation

1.12.2 Document and Version Control

Microsoft Teams will be used for task management and will be monitored regularly. All project artifacts will be maintained using version control and stored across GitHub, Google Drive, and Microsoft Teams to ensure accessibility, collaboration, and proper tracking of changes. Version tracking practices, including commit history and file versioning, will be used to monitor modifications, while collaborative workflows will ensure that team members can contribute efficiently without conflicts.

CHAPTER 2: Literature Review & Existing Systems

This chapter provides an overview of the existing and related systems.

2.1 Introduction

The university timetabling problem is a well-known NP-hard combinatorial optimization problem that involves assigning courses, instructors, rooms, and timeslots while satisfying hard constraints and minimizing soft constraint violations. As the number of resources and constraints increases, the number of possible scheduling combinations grows exponentially, making the problem difficult to solve using exact methods [5].

To address this complexity, researchers have explored a wide range of heuristic and meta-heuristic techniques capable of producing feasible and high-quality timetables within acceptable computational time. In practice, however, many university scheduling systems rely on rule-based approaches and simple heuristics due to their reliability, ease of implementation, and transparency for non-technical users. While metaheuristic methods often achieve near-optimal solutions, their computational overhead, parameter sensitivity, and limited interpretability have restricted their widespread adoption in operational systems. Nevertheless, they remain a major focus of academic research due to their strong optimization capabilities.

This section reviews the main categories of optimization techniques applied to the timetabling problem, including heuristic methods, evolutionary algorithms, swarm intelligence approaches, and hybrid techniques.

2.1.1 Review of Optimization Approaches in Literature

1. Basic Heuristics

Heuristic methods are widely used in timetabling due to their ability to generate feasible solutions efficiently without exploring the entire search space. These approaches rely on practical decision rules to guide the scheduling process, making them suitable for large-scale problems where exact methods are infeasible.

Qu et al. [6] provide a comprehensive survey of heuristic search methodologies in examination and course timetabling. Their work highlights the use of constructive heuristics, where scheduling decisions are made sequentially based on priority rules, such as assigning the most constrained courses first. While these methods are effective in quickly generating feasible timetables, their reliance on local decisions often limits the overall quality of the solution.

Similarly, Socha and Knowles [7] investigate search-based heuristic strategies that improve upon initial solutions through iterative refinement. Instead of constructing schedules from scratch, these methods explore neighboring solutions and make localized adjustments to reduce conflicts and improve timetable quality. Compared to constructive heuristics, this approach

provides a better balance between efficiency and solution quality.

Overall, heuristic methods remain highly relevant in practical applications due to their simplicity and efficiency. However, their limited ability to escape local optima motivates the use of more advanced optimization techniques.

II. Evolutionary Algorithms

Evolutionary Algorithms (EAs) are optimization techniques inspired by natural evolution. They operate by generating a population of candidate solutions and iteratively improving them through operators such as selection, crossover, and mutation. These methods are particularly suitable for solving the University Course Timetabling Problem due to its large and complex search space [8].

Genetic Algorithms (GA) are among the most widely used evolutionary approaches in timetabling. In GA, each timetable is represented as a chromosome, and a fitness function evaluates how well it satisfies constraints. Rodríguez-Maya et al. [8] compared multiple EAs and showed that GA can effectively generate feasible timetables, although performance depends on encoding strategies and parameter tuning. Similarly, Kakkar et al. [10] demonstrated that GA-based methods can successfully assign courses, rooms, and timeslots while minimizing conflicts.

Differential Evolution (DE) generates new candidate solutions by combining existing ones through weighted differences. Although originally designed for continuous optimization, it has been adapted for discrete problems such as timetabling. Shaker et al. [9] applied DE to the timetabling problem and reported high-quality solutions with reduced constraint violations. Comparative studies indicate that DE can outperform GA in certain scenarios, particularly in terms of solution quality [8].

Evolution Strategies (ES) focus primarily on mutation and self-adaptation rather than crossover. While ES has been applied less frequently to timetabling, some studies have shown its ability to generate feasible solutions [11]. However, its performance has not been as extensively evaluated as GA or DE.

Overall, evolutionary algorithms provide flexible and powerful approaches for solving complex timetabling problems, with GA being the most widely adopted and DE showing competitive performance.

III. Swarm Intelligence

Swarm intelligence algorithms are inspired by the collective behavior of natural systems and have been widely applied to timetabling problems due to their effectiveness in exploring large search spaces.

Particle Swarm Optimization (PSO) is one of the most commonly used swarm-based techniques. Tassopoulos et al. [1] proposed a local PSO-based algorithm and demonstrated improved solution quality, stability, and execution time compared to standard PSO, particularly for complex scheduling instances. Other studies have also shown that PSO can generate conflict-free

timetables efficiently [4].

Ant Colony Optimization (ACO) has also been successfully applied to timetabling problems. ACO constructs solutions using pheromone-guided search mechanisms, enabling effective handling of complex constraints. Studies have shown that ACO-based approaches produce high-quality timetables with performance comparable to other metaheuristic methods [3, 12].

More recent research has explored alternative swarm-based techniques such as the Whale Optimization Algorithm (WOA). Austero et al. [2] applied WOA to the course timetabling problem and demonstrated its ability to generate feasible schedules. However, the method exhibited higher computational time for larger datasets, which has led to the development of hybrid approaches that improve convergence speed and solution quality.

Overall, swarm intelligence methods have proven effective for timetabling, especially when enhanced with hybrid or localized strategies.

IV. Hybrid Algorithms

Hybrid algorithms combine multiple optimization techniques to leverage their respective strengths, typically integrating global exploration methods with local refinement strategies. This approach aims to improve both solution quality and computational efficiency.

Fen et al. [25] proposed a hybrid Particle Swarm Optimization combined with Constraint-Based Reasoning (PSO-CBR) approach for university timetabling. By incorporating domain-specific constraints directly into the optimization process, the method reduces the search space and improves solution feasibility.

Similarly, Badoni and Gupta [26] introduced a hybrid model combining Genetic Algorithms with Iterated Local Search (ILS). In this approach, the genetic algorithm performs global exploration, while local search refines promising solutions. Their results demonstrate improved timetable quality and reduced likelihood of convergence to suboptimal solutions.

More recent work by Banda and Halubanza [27] presents a hybrid system integrating genetic algorithms with Tabu search for examination timetabling. Their system significantly reduced scheduling conflicts and improved overall efficiency, highlighting the effectiveness of combining global and local optimization strategies.

These studies demonstrate that hybrid approaches can effectively balance exploration and exploitation, making them highly suitable for complex timetabling problems. Motivated by these findings, this project adopts a flexible approach that enables multiple optimization algorithms to be implemented and compared within a unified system, rather than relying on a single method.

2.2 Existing Systems

Over the years, several systems have been developed to make university timetabling easier. Some are free and open-source, others are commercial, and many focus more on administrative needs than student-centered scheduling. The following are some commonly used timetabling tools and systems.

UniTime [15] is a well-known open-source university timetabling system that is free to use, making it suitable for large universities or research-focused institutions with dedicated IT staff. It facilitates scheduling for courses, exams, and student sectioning, even accommodating complex multi-campus setups. The system employs various algorithms such as constraint-based optimization, local search, hill-climbing, conflict-based statistics, and the Great Deluge metaheuristic. Its strengths include the ability to manage very complex schedules while supporting instructor availability and room constraints, all within an actively maintained open-source framework. However, the installation and configuration can prove challenging, often requiring technical knowledge, and it has limited features for optimizing student-centered schedules.

aSc Timetables [16] is a commercial timetabling tool that requires a paid license, designed for schools and smaller higher-education institutions, focusing on speed and ease of use. It enables users to generate timetables quickly with minimal technical knowledge, utilizing proprietary heuristic/evolutionary optimization algorithms. Its strengths include fast timetable generation, a user-friendly interface, effective handling of basic constraints, and the ability to produce printable schedules. However, it has some weaknesses, such as being closed-source and paid, providing limited support for complex university-level constraints, and lacking student-centered schedule optimization.

FET (Free Timetabling Software) [17] is a free, open-source tool that is moderately easy to use, making it suitable for small to medium-sized educational institutions or for research and experimentation. It offers a flexible system that allows users to modify constraints and test various scheduling scenarios. Its strengths include being open-source, flexible, and cross-platform, with a transparent logic that supports experimentation. However, it has limited scalability for very large universities, a basic and somewhat outdated interface, and restricted features for student scheduling.

Scientia Syllabus Plus [18] is a leading commercial scheduling system tailored for large universities, adept at managing extensive datasets and complex constraints. Employing advanced algorithms like constraint programming and mathematical optimization, it effectively addresses intricate scheduling challenges. The system's strength lies in its power, scalability, and dedicated professional support. However, its high licensing costs can be prohibitive for smaller institutions, and being a closed-source solution, it lacks customization options.

University Internal Scheduling Systems such as Banner, PeopleSoft Campus Solutions, Moodle-based registration modules, and SIS systems are easy for students and administrators but generally do not perform automatic optimization. Their strengths include integration with university databases, real-time conflict detection, automatic enforcement of prerequisites, and ease of use. However, they fail to provide an automatically optimized solution for scheduling and are inefficient in assigning resources such as rooms and timeslots.

Table 2.1: Summary of Existing Tools

Tool	Algorithm	Pros	Cons
UniTime	Constraint-based optimization; local search and hill-climbing; conflict-based statistics; Great Deluge metaheuristic	Can handle very complex schedules; supports instructor availability and room constraints; open-source and actively maintained	Difficult installation and configuration; requires technical knowledge; limited student-centered schedule optimization
aSc Timetables	Proprietary heuristic/evolutionary optimization	Fast timetable generation; user-friendly interface; handles basic constraints effectively; can produce printable schedules	Closed-source and paid; limited support for complex university-level constraints; no student-centered schedule optimization
FET	Heuristic and evolutionary-based approaches with constraint checking	Open-source; flexible; cross-platform support; transparent logic for experimentation; good for teaching or research	Less scalable for very large universities; interface is basic and outdated; limited student scheduling features
Scientia Syllabus Plus	CSP and ILP-like proprietary techniques	Very powerful and scalable; handles complex constraints efficiently; includes professional support and reporting tools	Expensive; closed-source; not customizable for research; limited student-centered features
Internal Scheduling Systems	Rule-based validation and conflict checking	Integrated with university databases; real-time conflict detection; easy for students to use; automatically enforces prerequisites	No automatic optimization; students must manually create schedules; instructor preferences often ignored; inefficient use of rooms and time slots

2.2.1 Overall Problems of Existing Systems

Most existing scheduling tools focus more on administrative needs than student optimization. Open-source systems like UniTime and FET are flexible and transparent but can be hard to set up and configure. Commercial systems such as aSc Timetables and Scientia Syllabus Plus are user-friendly but expensive and closed-source. Internal university systems are easy for students but do not automatically optimize schedules. Overall, there is a need for systems that balance institutional constraints with student preferences while remaining practical and accessible.

2.3 Overall Solution Approach

Despite the extensive research on optimization techniques for the university timetabling problem, most studies focus on evaluating a single algorithm or a specific hybrid approach in isolation. There is limited emphasis on developing flexible systems that allow multiple algorithms to be implemented, tested, and compared within the same environment.

Furthermore, while advanced metaheuristic methods demonstrate strong optimization performance, real-world scheduling systems often prioritize simplicity, reliability, and ease of use over computational complexity. This creates a gap between research-oriented optimization techniques and practical implementation.

Therefore, there is a need for a system that bridges this gap by supporting multiple optimization approaches while maintaining usability and practical applicability. This project addresses this need by developing a web-based timetabling system that enables the comparison of different algorithms under a unified framework.

The solution is a smart scheduling approach that balances both administrative requirements and student needs. It uses scheduling algorithms to generate efficient and optimized timetables. The system is designed to be easy to use while still supporting complex constraints like room capacity, instructor availability, and campus rules. It also allows students and instructors to set their preferences, which are considered during schedule generation.

In addition, the system gives flexibility by allowing manual adjustments after automatic generation, ensuring practical and realistic schedules. By integrating optimization, user preferences, and a simple interface, the system provides a more balanced and accessible scheduling solution compared to existing tools.

CHAPTER 3: Problem Formulation

This chapter presents the mathematical formulation of the problem.

3.1 Sets

The model is defined over the following sets:

Table 3.1: Sets

Symbol	Description	Index
$S = \{1, 2, \dots, S \}$	Set of course sections	s
$R = \{1, 2, \dots, R \}$	Set of rooms	r
$T = \{1, 2, \dots, T \}$	Set of timeslots	t
$I = \{1, 2, \dots, I \}$	Set of instructors	i
$D = \{1, 2, \dots, D \}$	Set of departments	d

In this formulation, each section s represents a predefined triplet of (`course_id`, `instructor_id`, `section_number`) derived from the university’s base schedule. The assignment of instructors to courses is treated as fixed input data rather than a decision variable. Consequently, the optimization problem reduces to assigning each pre-paired section to a feasible combination of room r and timeslot t . Instructor identity is retained as a section attribute solely for the purpose of enforcing the no double-booking constraint C2, ensuring that no two sections sharing an instructor are assigned to the same timeslot.

The following parameters are known inputs to the model:

Table 3.2: Parameters

Parameter	Description
c_s	Enrollment capacity (required seats) of section s
cap_r	Seating capacity of room r
τ_s	Course type of section s (e.g., lecture, lab, seminar)
ρ_r	Room type of room r (e.g., lecture hall, lab, seminar room)
ι_s	Instructor assigned to section s (may be null)
δ_s	Department of section s
$A_{s,t}$	Binary: 1 if section s is statically allowed in timeslot t , 0 otherwise
$B_{s,r}$	Binary: 1 if room r is viable for section s (type + capacity match), 0 otherwise

3.2 Decision Variables

Since instructor-to-course assignment is fixed in the input data, the primary binary decision variable is:

$$x_{s,r,t} = 1 \text{ if section } s \text{ is assigned to room } r \text{ in timeslot } t; 0 \text{ otherwise} \quad \forall s \in S, r \in R, t \in T$$

An auxiliary (derived) variable indicates whether a section is scheduled at all:

$$y_s = \sum_{r \in R} \sum_{t \in T} x_{s,r,t} \in \{0, 1\}$$

3.3 Objective Function

We maximize a weighted fitness function F , which considers assignment completeness, room utilization, and penalties for conflicts, unassigned sections, and uneven department load distribution:

$$\max F(X) = w_1 \cdot \Phi_{\text{assign}} + w_2 \cdot \Phi_{\text{util}} - w_3 \cdot \Phi_{\text{conf}} - w_4 \cdot \Phi_{\text{wait}} - w_5 \cdot \Phi_{\text{spread}}$$

(a) Assignment Completeness — Rewards schedules that successfully place more sections:

$$\Phi_{\text{assign}} = \frac{1}{|S|} \sum_{s \in S} y_s$$

(b) Room Utilization — Rewards a tight fit between section size and room capacity (avoids tiny classes in huge rooms):

$$\Phi_{\text{util}} = \frac{1}{|S|} \sum_{s,r,t} \frac{c_s}{\text{cap}_r} \cdot x_{s,r,t}$$

(c) Soft Conflict Penalty — Penalizes instructor overloads that escape hard constraint enforcement:

$$\Phi_{\text{conf}} = \sum_{i \in I} \sum_{t \in T} \max \left(0, \sum_{s: \iota_s = i, r} x_{s,r,t} - 1 \right)$$

(d) Unassigned Penalty — Penalizes each section that remains unscheduled:

$$\Phi_{\text{wait}} = \sum_{s \in S} (1 - y_s)$$

(e) Department Spread — Penalizes uneven distribution of each department's sections across timeslots:

$$\Phi_{\text{spread}} = \sum_{d \in D} \text{Var}_t \left(\sum_{s: \delta_s = d, r} x_{s,r,t} \right)$$

3.4 Constraints

3.4.1 Hard Constraints

Table 3.3: General Hard Constraints

Constraint
(C1) At most one assignment per section
(C2) No instructor double-booking
(C3) No room double-booking
(C4) Room capacity sufficiency
(C5) Room-type compatibility
(C6) Valid timeslot
(C7) Viable room

(C1) At most one assignment per section

$$\sum_{r \in R} \sum_{t \in T} x_{s,r,t} \leq 1 \quad \forall s \in S$$

(C2) No instructor double-booking — An instructor cannot teach two sections at the same

time:

$$\sum_{s: \iota_s=i} \sum_{r \in R} x_{s,r,t} \leq 1 \quad \forall i \in I, \forall t \in T$$

(C3) No room double-booking — A room cannot host two sections at the same time:

$$\sum_{s \in S} x_{s,r,t} \leq 1 \quad \forall r \in R, \forall t \in T$$

(C4) Room capacity sufficiency — The assigned room must seat the enrolled students:

$$c_s \cdot x_{s,r,t} \leq \text{cap}_r \quad \forall s, r, t$$

(C5) Room-type compatibility — Lab courses must be assigned to lab rooms, lectures to lecture halls, etc.:

$$x_{s,r,t} \leq \mathbf{1}[\tau_s = \rho_r] \quad \forall s, r, t$$

(C6) Valid timeslot:

$$x_{s,r,t} \leq A_{s,t} \quad \forall s, r, t$$

(C7) Viable room (combines C4 and C5), derived from `get_viable_rooms`:

$$x_{s,r,t} \leq B_{s,r} \quad \forall s, r, t$$

Table 3.4: Hard Constraints for Dataset

Constraint
(C8) Department Constraint
(C9) Campus Constraint
(C10) Timeslot Validity Constraint

(C8) Department Constraint — Rooms assigned to a specific department can only be used by courses from the same department.

(C9) Campus Constraint — Sections must be scheduled in rooms located on the same campus. Campus variations include: Main campus, Women campus, Men campus, Medical campus.

(C10) Timeslot Validity Constraint — Each section must be assigned to a valid timeslot based on its type, duration, and scheduling rules. Examples:

- Lectures: scheduled on MW or TR
- Labs: scheduled on single days
- Intensive courses: scheduled in extended-duration blocks

3.4.2 Soft Constraints

Soft constraints improve timetable quality but are not strictly required.

Table 3.5: Soft Constraints

Constraint
(C11) Minimize Gaps Constraint
(C12) Scheduling Priority Constraint
(C13) Section Linking Constraint

(C11) Minimize Gaps Constraint — Gaps between courses are preferred to be minimized.

(C12) Scheduling Priority Constraint — Certain course types (e.g., office hours and projects) are preferred to be scheduled later in the day, giving priority to lectures and labs.

(C13) Section Linking Constraint — Related sections (e.g., lecture and lab pairs) should follow logical scheduling relationships.

3.5 Boundary Conditions

The following bounds apply to the decision variables, parameters, and GA hyperparameters:

(B1) Binary decision variables:

$$x_{s,r,t} \in \{0, 1\} \quad \forall s \in S, r \in R, t \in T$$

(B2) Parameter bounds:

$$c_s \geq 1, \quad \text{cap}_r \geq 1, \quad |S|, |R|, |T| \in \mathbb{Z}^+$$

(B3) GA hyperparameter bounds: Details in Chapter 6.

3.6 Compact Problem Statement

The complete optimization problem can be stated compactly as:

$$\max_X F(X) = w_1 \Phi_{\text{assign}} + w_2 \Phi_{\text{util}} - w_3 \Phi_{\text{conf}} - w_4 \Phi_{\text{wait}} - w_5 \Phi_{\text{spread}}$$

subject to:

$$\begin{aligned}
\sum_{r,t} x_{s,r,t} &\leq 1 && \forall s \\
\sum_{s: l_s=i, r} x_{s,r,t} &\leq 1 && \forall i, t \\
\sum_s x_{s,r,t} &\leq 1 && \forall r, t \\
x_{s,r,t} &\leq A_{s,t} \cdot B_{s,r} && \forall s, r, t \\
c_s \cdot x_{s,r,t} &\leq \text{cap}_r && \forall s, r, t \\
x_{s,r,t} &\in \{0, 1\} && \forall s, r, t
\end{aligned}$$

The weights w_1 through w_5 define the relative importance of each objective component. In the implemented system, this multi-component objective is operationalized through a two-term penalty function that concentrates optimization pressure on the two most impactful components: assignment completeness (corresponding to $w_4 \cdot \Phi_{\text{wait}}$) and conflict minimization (corresponding to $w_3 \cdot \Phi_{\text{conf}}$). The remaining components (room utilization, soft conflict penalties, and department spread) are addressed through the constraint management and room selection modules. The mapping between the theoretical formulation and the implemented fitness function is described in detail in Section 6.7.

CHAPTER 4: Requirement Engineering and Analysis

4.1 Stakeholders

This project includes different stakeholders who may use the system or be affected by it.

I. Primary Stakeholders — These are the main users who directly use the system:

- – **Administrator / Registration Department:** Manages the system, updates data, organizes course schedules, assigns sections, and runs scheduling algorithms.
- **Students:** Use the system to view and create their schedules.
- **Instructors:** View their teaching schedules and set preferences such as preferred time slots.

II. Secondary Stakeholders — These stakeholders do not use the system directly but are still affected by it:

- – **University Management:** Uses the schedules for high-level planning and decision-making (e.g., Dean, Head of Department).
- **IT Support Team:** Maintains the system and fixes technical problems when needed.

III. External Stakeholders — These are outside the university but still connected to the system:

- – **Education Authorities:** Organizations that check the quality and standards of the university.
- **Future Students:** Benefit from better organized schedules and course availability.

4.2 Use Case Diagram

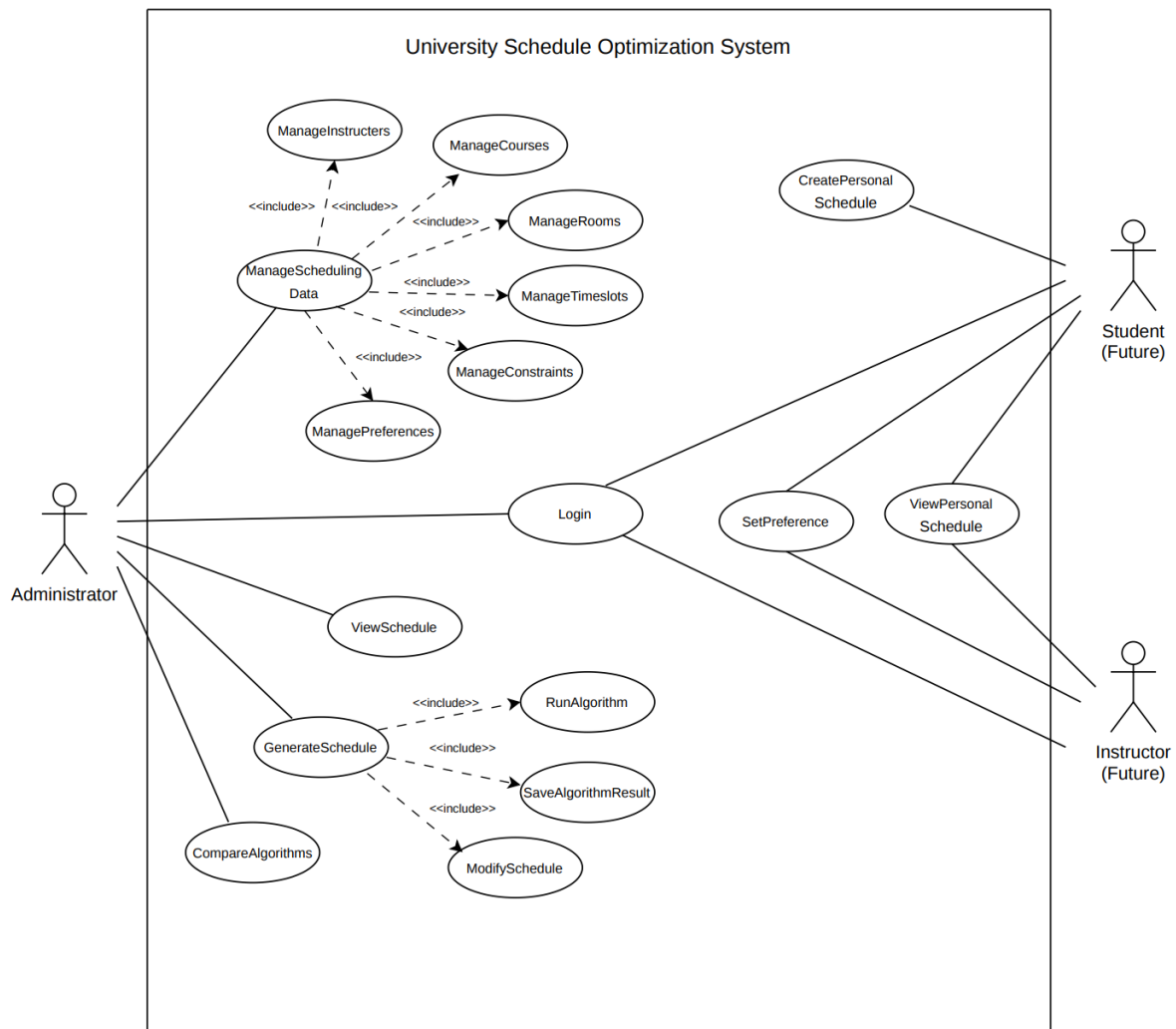


Figure 4.1: Use Case Diagram

4.2.1 Use Case Description

Each use case has two tables. The first table describes the primary actors, preconditions, and postconditions of each use case along with the normal flow. The second table includes the alternative flow, representing situations where the process does not follow the normal path.

Table 4.1: Use Case of Login

Primary Actor	Preconditions	Postconditions
User (Administrator, Instructor, Student)	User must have a registered account in the system	User is authenticated and redirected to the dashboard; OR login is rejected
Normal Flow		
1. User enters email and password.		
2. System sends credentials to the authentication module.		
3. System retrieves user data from the database.		
4. System validates credentials.		
5. System grants access.		
6. Dashboard is displayed.		

Table 4.2: Login Alternative Flows

Alternative Flow	Description
A1 Invalid Credentials	System fails to validate username/password; error message displayed: “Invalid credentials”; user is asked to try again.
A2 Empty Fields	User submits form with missing fields; system prompts: “Please fill all required fields”.
A3 User Does Not Exist	No matching account found in the system; login is denied; error message: “User does not exist”.

Table 4.3: Use Case of Manage Scheduling Data

Primary Actor	Preconditions	Postconditions
Administrator	Admin is logged in	Scheduling data is updated in system
Normal Flow		
1. Admin selects manage scheduling data.		
2. System provides options: Manage instructors; Manage courses; Manage rooms; Manage timeslots; Manage constraints; Manage preferences.		
3. Admin updates required data.		
4. System saves updates.		

Table 4.4: Manage Scheduling Data Alternative Flows

Alternative Flow	Description
A1 Invalid Data Entry	System detects incorrect or incomplete input; displays validation error message; requests the administrator to enter valid data.
A2 Save Failure	System fails to save the data (e.g., database error); displays error message: “Unable to save data”; administrator is asked to try again later.

Table 4.5: Use Case of Generate Schedule

Primary Actor	Preconditions	Postconditions
Administrator	All scheduling data is available; constraints and preferences are defined; at least one scheduling algorithm exists	Schedule is generated, optimized, and stored; administrator can view and modify the schedule

Normal Flow

1. Admin selects scheduling algorithm.
2. Admin requests schedule generation.
3. System runs the selected algorithm.
4. Algorithm processes constraints and data.
5. Algorithm applies optimization to improve schedule quality (e.g., minimize gaps, avoid conflicts, balance days).
6. System saves the generated schedule.
7. System displays the generated schedule.
8. Admin may modify the schedule, and the changes are applied and saved.

Table 4.6: Generate Timetable Alternative Flows

Alternative Flow	Description
A1 No Algorithm Selected	Administrator does not select an algorithm; system prompts the administrator to choose an algorithm before proceeding.
A2 No Algorithms Available	No algorithms exist in the system; system displays message: “No algorithm available”.
A3 Missing / Incomplete Data	Required data (courses, instructors, rooms) is missing; system stops execution and displays an error message.

Table 4.7: Use Case of Compare Algorithms

Primary Actor	Preconditions	Postconditions
Administrator	Multiple schedules exist; algorithms have been executed	Comparison results displayed

Normal Flow

1. Admin selects compare algorithms.
2. System retrieves all schedules from the database.
3. System sends schedules to the algorithm engine.
4. Metrics are calculated (e.g., conflicts, time).
5. System returns comparison data.
6. Results are displayed.

Table 4.8: Compare Algorithms Alternative Flows

Alternative Flow	Description
A1 No Results Available	No timetable results are stored in the system; the system displays: “No schedules to compare.”
A2 Incomplete Data	Some algorithm results are missing or not fully generated; the system displays available comparison results and notifies the administrator about missing data.
A3 Execution Error	An error occurs during algorithm execution or comparison; the system displays an error message and stops the process.

Table 4.9: Use Case of View Schedule

Primary Actor	Preconditions	Postconditions
Administrator	Schedule exists	Schedule is displayed
Normal Flow		
1. Admin requests to view the schedule.		
2. System fetches schedule from database.		
3. System displays schedule.		

Table 4.10: View Timetable Alternative Flows

Alternative Flow	Description
A1 No Timetable Available	No timetable exists in the system; system returns an empty result; message displayed: “No schedule available”.

Figure 4.1 presents the complete use case diagram for the system. The use cases associated with the Student and Instructor actors are documented for future planning purposes but are not implemented in the current phase. The following section describes each use case in detail.

Table 4.11: Use Case of Set Preference

Primary Actor	Preconditions	Postconditions
Student AND Instructor	User is logged in	Preferences saved
Normal Flow		
1. User selects set preference.		
2. User enters preferences.		
3. System validates input.		
4. System stores preferences.		

Table 4.12: Set Preference Alternative Flows

Alternative Flow	Description
A1 Invalid Preference	System detects invalid or unsupported preference input; input is rejected; error message is displayed to the user.

Table 4.13: Use Case of View Personal Schedule

Primary Actor	Preconditions	Postconditions
Student AND Instructor	Personal schedule exists	Personal schedule displayed
Normal Flow		
1. User requests personal schedule.		
2. System fetches schedule using user ID.		
3. System displays schedule.		

Table 4.14: View Personal Schedule Alternative Flows

Alternative Flow	Description
A1 No Personal Schedule	No schedule exists for the user; system returns empty result; message displayed: "No personal schedule available".

Table 4.15: Use Case of Create Personal Schedule

Primary Actor	Preconditions	Postconditions
Student	Preferences are set; classes and schedule data exist in the system	Personalized schedule is created, optimized, and stored; schedule is displayed to the student
Normal Flow		
1. Student requests schedule creation.		
2. System retrieves class details based on preferences.		
3. System validates the schedule (checks for conflicts, prerequisites, and capacity).		
4. System applies optimization to improve schedule quality (e.g., minimize gaps, balance days).		
5. System saves the personalized schedule.		
6. System displays the generated schedule.		

Table 4.16: Create Personal Timetable Alternative Flows

Alternative Flow	Description
A1 Preferences Not Set	Student has not set the required preferences; system prompts the student to set preferences before proceeding.
A2 Preference Conflict	System cannot satisfy all selected preferences; system suggests an alternative schedule or asks the student to modify preferences.
A3 Invalid Class Selection	Selected class does not exist or is already full; system displays an error message and prompts for a new selection.

4.3 Non-Functional Requirements

1. **Performance:** The system shall provide fast and consistent response times for all user operations and optimization processes.
2. **Scalability:** The system shall support growth in the number of users, courses, instructors, rooms, and students without performance degradation.
3. **Usability:** The system shall provide a clear, intuitive, and user-friendly web interface that requires minimal training.

4. **Reliability:** The system shall operate consistently without unexpected failures and shall ensure data accuracy and consistency across all operations.
5. **Security:** The system shall restrict access to authorized users only and protect sensitive data from unauthorized access or modification.
6. **Maintainability:** The system shall follow a modular and well-documented design to allow easy maintenance, debugging, and future enhancements.
7. **Availability:** The system shall be accessible to authorized users whenever required and minimize system downtime.
8. **Data Integrity:** The system shall validate all input data to ensure accuracy and shall prevent duplication, corruption, or inconsistent records in the database.
9. **Portability:** The system shall operate on modern web browsers and standard server environments without major configuration changes.
10. **Extensibility:** The system shall be designed to support future integration of advanced optimization techniques and student-centered scheduling features.

4.4 Requirements Traceability Matrix

Table 4.17: List of Functional Requirements

Req ID	Requirement Description	Design Component	Test Case ID	Status
FR-01	Allow admin users to define courses, instructors, rooms, time slots, and constraints	Data Management Module	TC-01	Pass
FR-02	Generate feasible timetables satisfying hard constraints using scheduling algorithms	Scheduling Engine	TC-02	Pass
FR-03	Support execution of multiple scheduling algorithms for comparison	Algorithm Module	TC-03	Pass
FR-04	Allow admins to modify generated timetables	User Interface / Editor Module	TC-04	Pass
FR-05	Detect and prevent instructor and room conflicts	Constraint Handling Module	TC-05	Pass
FR-06	Compute evaluation metrics (conflicts, gaps, execution time)	Evaluation Module	TC-06	Pass
FR-07	Display timetables in a clear tabular format	Visualization Module	TC-07	Pass
FR-08	Validate user input and provide feedback if generation fails	Validation Module	TC-08	Pass
FR-09	Provide role-based access for instructors and students (FUTURE)	Authentication & Authorization Module	TC-09	Future
FR-10	Generate student schedules based on selected courses (FUTURE)	Student Scheduling Module	TC-10	Future

CHAPTER 5: Architecture and Design

5.1 Overview

This chapter describes the system design components and architecture design, including system architecture design, UML diagrams, the database ER diagram, and the user interface prototype.

5.2 Software Architecture

5.2.1 Architecture Style

The system adopts a layered architecture style, which organizes the system into multiple layers, each responsible for a specific set of tasks. This architectural approach separates user interaction, system processing, optimization logic, and data management into independent components. Such separation improves modularity, simplifies system maintenance, and allows future enhancements to be implemented with minimal impact on other parts of the system.

In this design, the system is structured into four main layers: the Presentation Layer, Application Layer, Optimization Layer, and Data Layer.

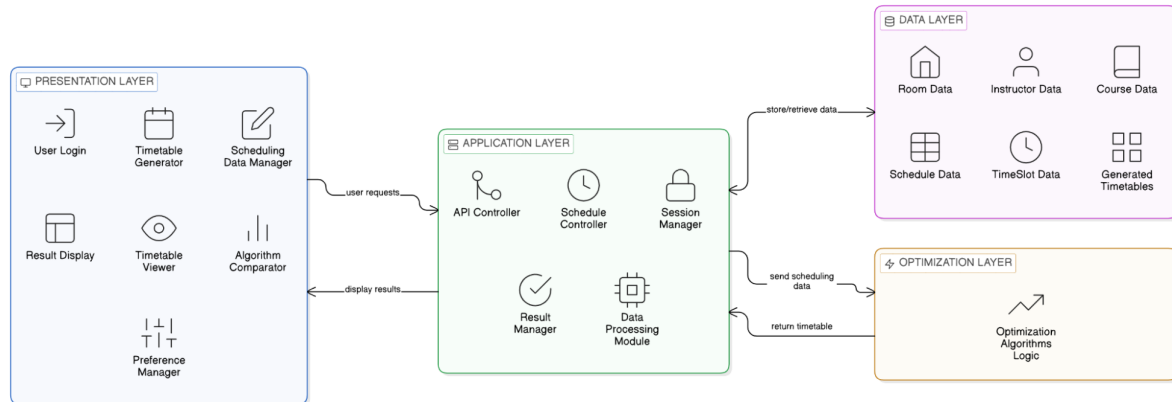


Figure 5.1: Layered Architecture Diagram

5.2.2 Architecture Components

Presentation Layer is the frontend of the system and handles all interactions between the user and the system. It provides interfaces for administrators, students, and instructors to input scheduling data, request timetable generation, and view the generated results.

Application Layer implements the core business logic and coordinates communication between the user interface, the optimization logic, and the database. It ensures that requests from the presentation layer are validated, processed, and correctly forwarded for timetable generation. The API controller serves as the main entry point for user requests. The Session Manager oversees user sessions and authentication. The Data Processing Module validates and prepares

scheduling data for optimization. The Schedule Controller coordinates the timetable generation workflow. The Result Manager collects timetables from the optimization layer, formats them, and returns the results to the presentation layer.

Optimization Layer is responsible for processing the scheduling problem and generating an optimized timetable. This layer encapsulates the core logic for applying optimization techniques, evaluating constraints, and producing feasible schedules.

Data Layer manages all persistent storage within the system. It ensures that scheduling data, generated timetables, and related system information are stored securely and can be retrieved efficiently when needed.

5.3 Software Design

5.3.1 UML Sequence / Communication Diagrams

1. Login Sequence Diagram

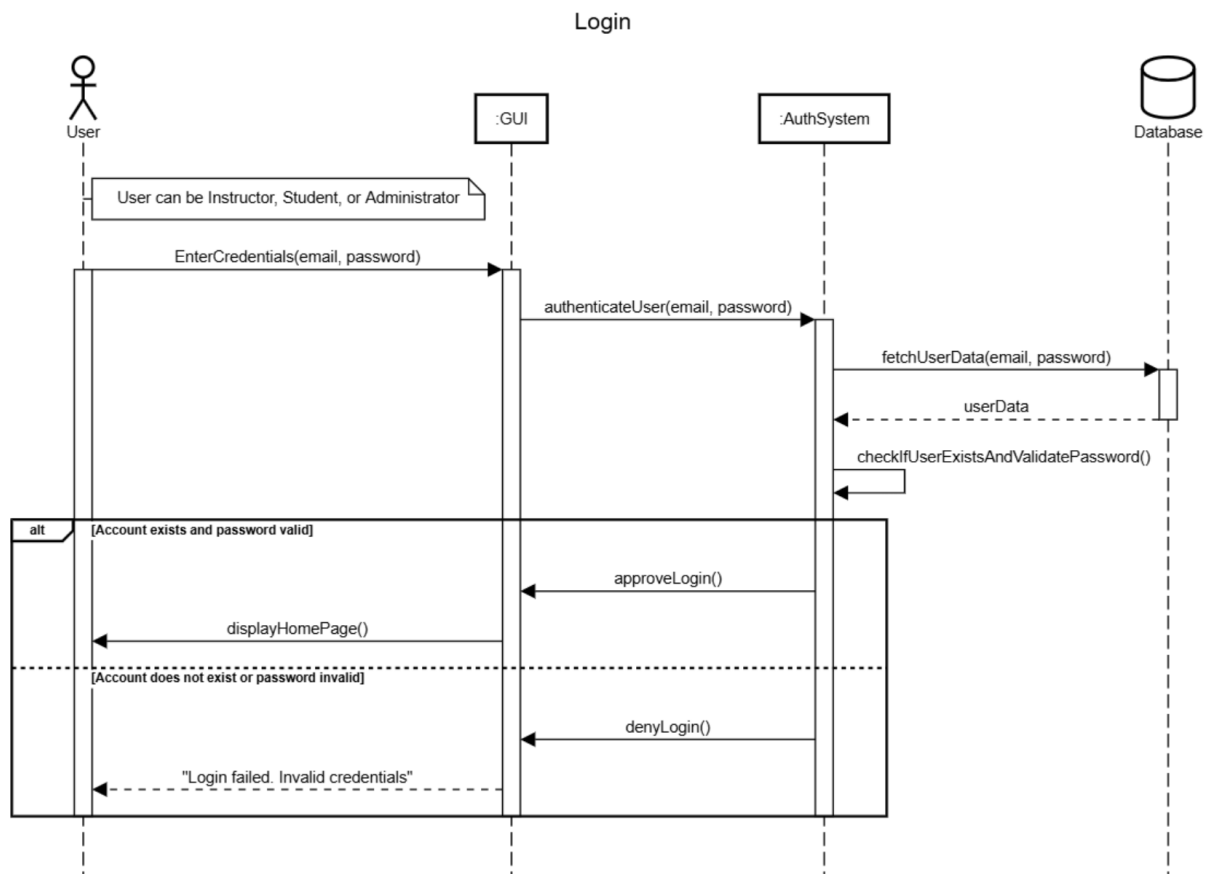


Figure 5.2: Login Sequence Diagram

This diagram shows the process of user authentication in the system. Objects involved:

- **User (Actor):** The person trying to log into the system (Student, Instructor, or Admin).
- **GUI:** Interface where the user enters credentials and receives feedback.
- **AuthSystem (Controller):** Handles login verification and authentication logic.

- **Database:** Stores user account information and credentials.

All interactions are **synchronous**, meaning each step must finish before the next one begins. The AuthSystem sends credentials to the database, receives a response, and then returns either a success or failure result to the user interface.

II. Manage Scheduling Data Sequence Diagram

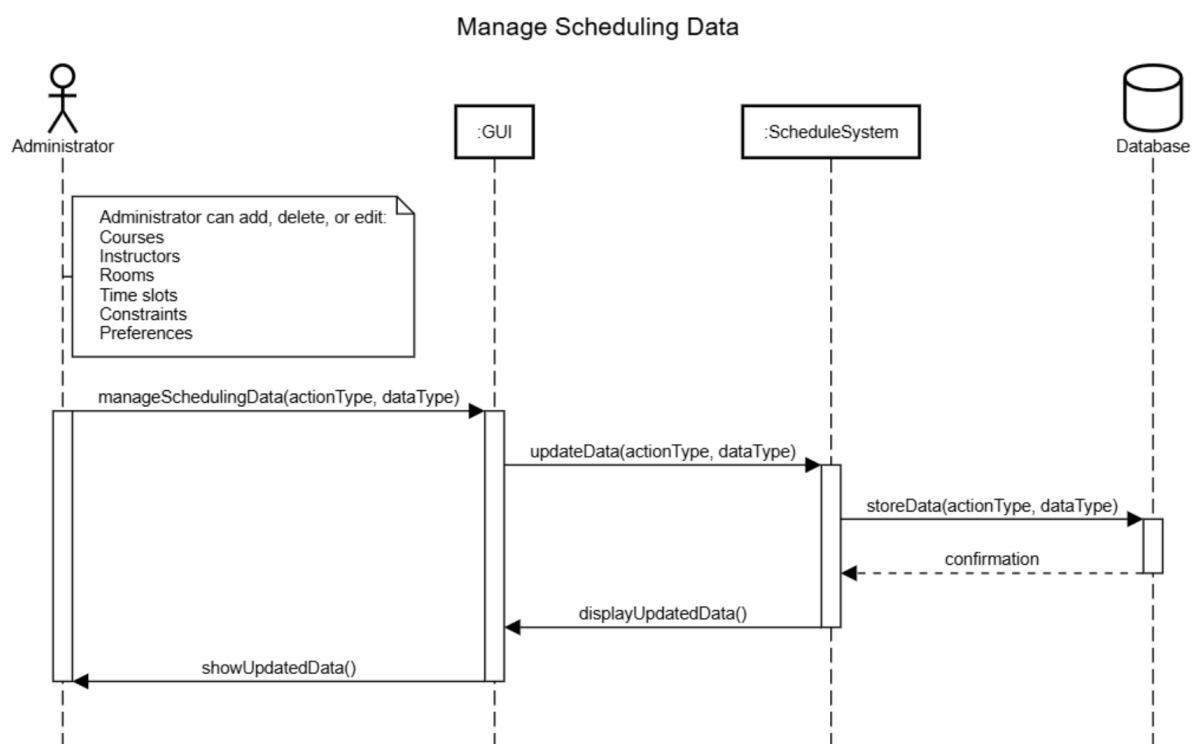


Figure 5.3: Manage Scheduling Data Sequence Diagram

This diagram shows how the administrator manages scheduling data. Objects involved:

- **Administrator (Actor):** The user responsible for managing system data.
- **GUI:** The interface used by the administrator to add, edit, or delete data.
- **ScheduleSystem (Controller):** Handles all logic for processing and updating scheduling data.
- **Database:** Stores all scheduling information permanently.

All interactions are **synchronous**. The flow goes from the GUI to the database through the controller, and the system returns confirmation once changes are saved successfully.

III. Generate Schedule Sequence Diagram

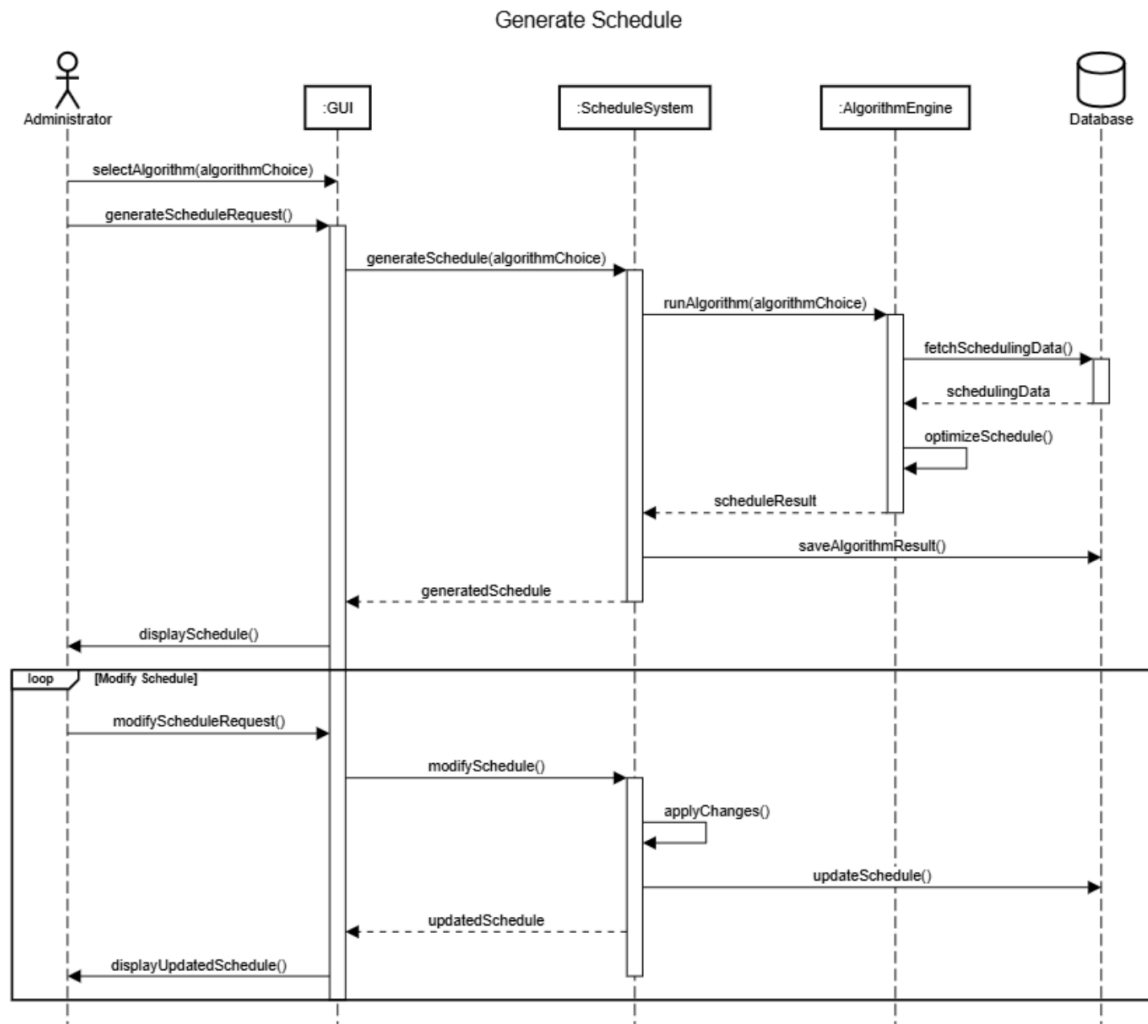


Figure 5.4: Generate Schedule Sequence Diagram

This diagram shows how the system generates a schedule using a selected algorithm. Objects involved:

- **Administrator (Actor):** Starts the schedule generation process and can manually modify the final schedule.
- **GUI (Boundary):** Interface used to select the algorithm and display results.
- **ScheduleSystem (Controller):** Manages communication between all components.
- **AlgorithmEngine:** Performs optimization and generates the best possible schedule.
- **Database:** Provides input data and stores the final schedule.

All interactions are **synchronous**. The system retrieves data from the database, runs the selected algorithm, and then stores the generated schedule. If manual changes are made, they are validated and updated in the database in the same sequential manner.

IV. Compare Algorithms Sequence Diagram

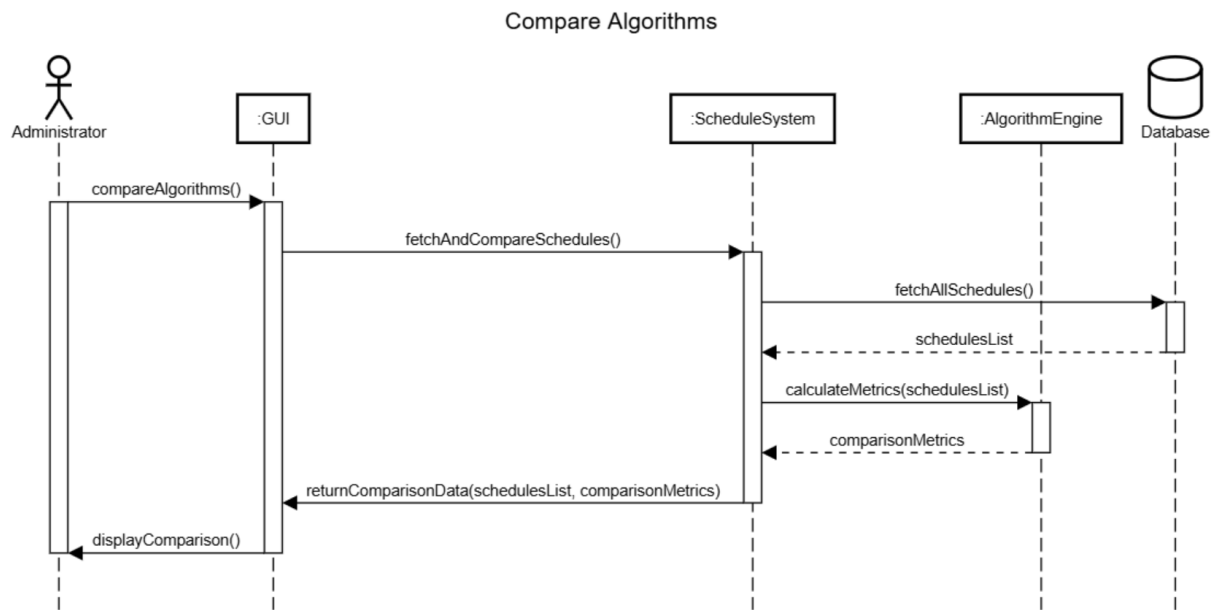


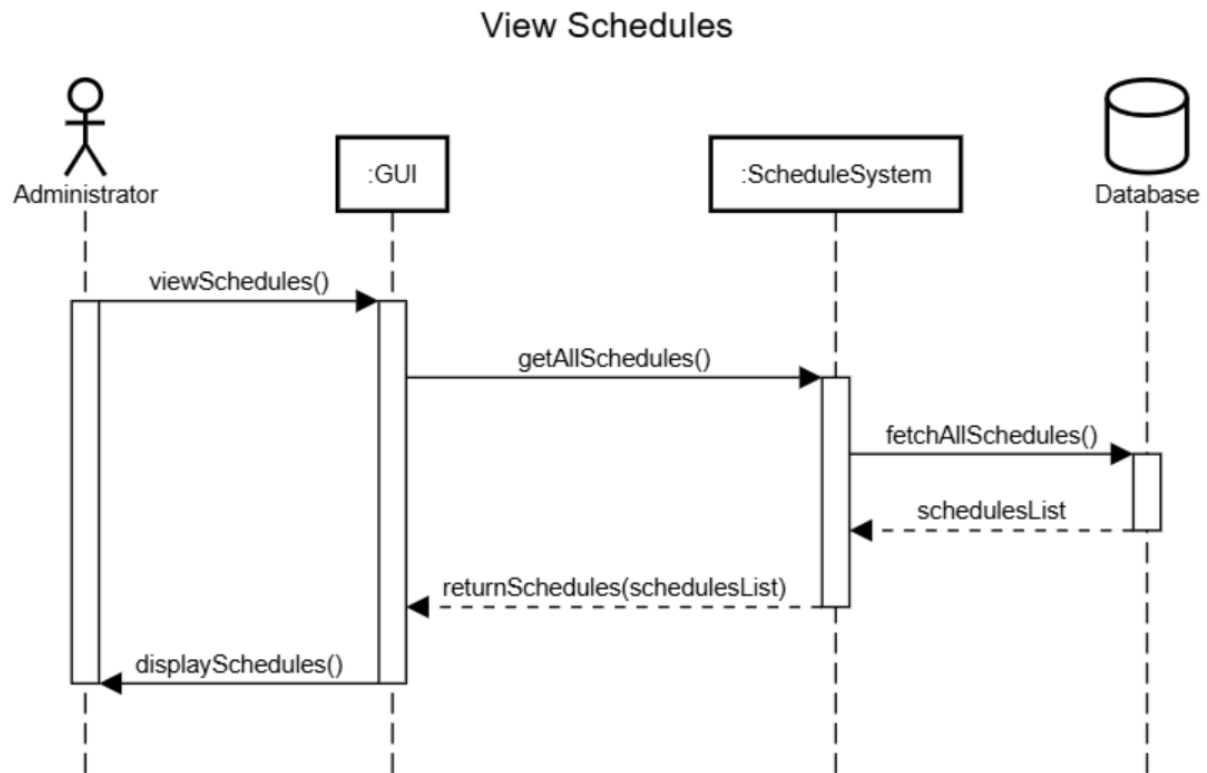
Figure 5.5: Compare Algorithms Sequence Diagram

This diagram shows how the Administrator compares different scheduling algorithms. Objects involved:

- **Administrator (Actor):** Starts the comparison process.
- **GUI:** Sends requests and shows results.
- **ScheduleSystem (Controller):** Manages system flow and communication.
- **AlgorithmEngine:** Calculates performance metrics.
- **Database:** Provides stored schedule data.

All interactions are **synchronous**. The system retrieves schedule data, processes it through the AlgorithmEngine, and then returns the computed comparison results back to the GUI.

V. View Schedules Sequence Diagram

**Figure 5.6:** View Schedules Sequence Diagram

This diagram shows how the Administrator retrieves and views all saved schedules from the system. It is a simple read-only process where data is fetched from the database and displayed on the interface. Objects involved:

- **Administrator (Actor):** Requests to view all schedules.
- **GUI:** Sends the request and displays the results.
- **ScheduleSystem (Controller):** Handles communication between GUI and database.
- **Database:** Stores all schedule records.

All interactions are **synchronous**. The system follows a request–response flow from the GUI to the database and back. The only return messages occur when the database sends back the stored schedule data.

VI. Set Preferences Sequence Diagram

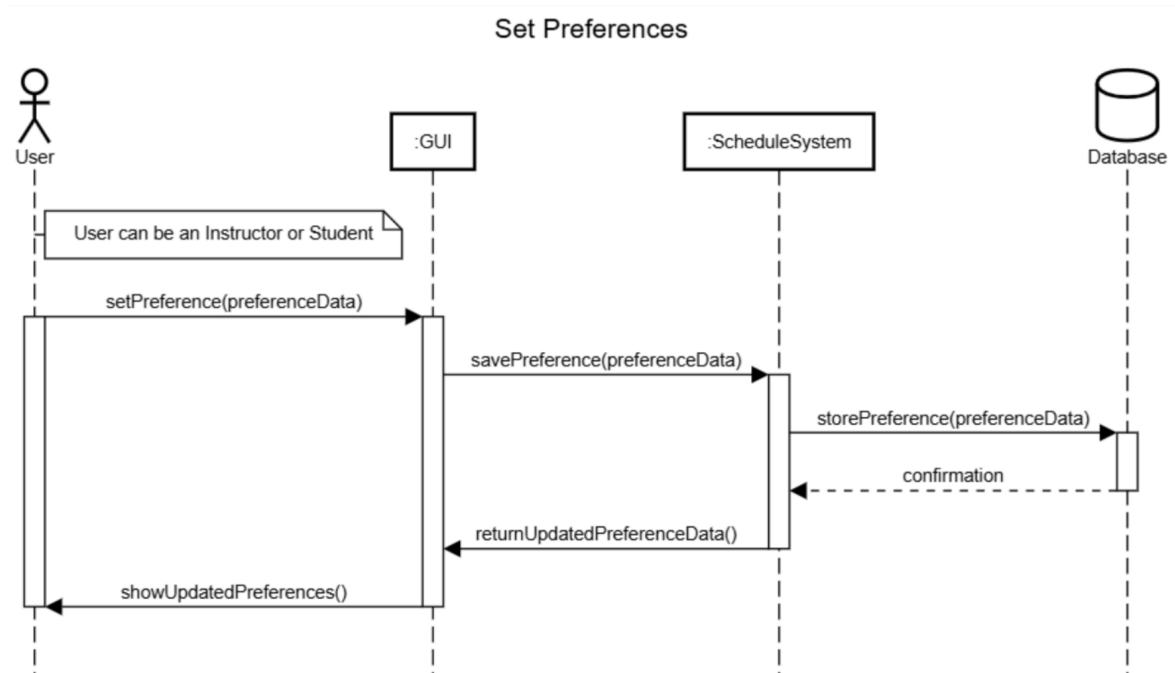


Figure 5.7: Set Preferences Sequence Diagram

This diagram shows how an Instructor or Student updates personal preferences in the system. Objects involved:

- **User (Actor):** Instructor or Student who sets preferences.
- **GUI:** Interface used to input preference data.
- **ScheduleSystem (Controller):** Handles validation and processing of preferences.
- **Database:** Stores user preference data permanently.

All interactions are **synchronous**. The system follows a request–response flow where preference data is sent from the user to the database through the controller, and confirmation is returned once the update is successfully completed.

VII. View Personal Schedule Sequence Diagram

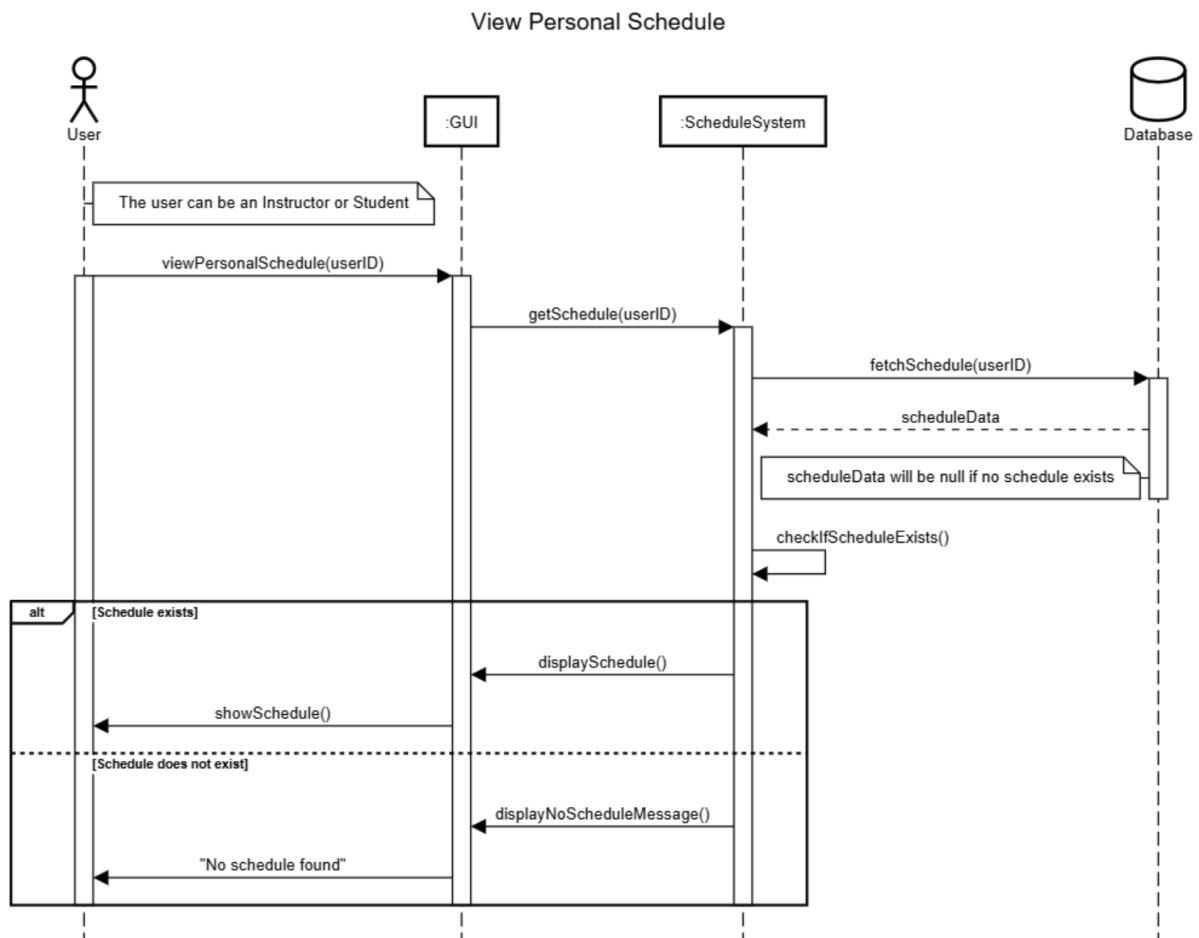


Figure 5.8: View Personal Schedule Sequence Diagram

This diagram shows how an Instructor or Student retrieves their personal timetable. The system checks whether a schedule exists for the given user ID and either displays the schedule or shows an error message if no data is found. Objects involved:

- **User (Actor):** Instructor or Student requesting their timetable.
- **GUI:** Receives the request and displays the result or error message.
- **ScheduleSystem (Controller):** Handles logic for retrieving and validating schedule data.
- **Database:** Stores all user-specific schedule records.

All interactions are **synchronous**. The system follows a strict request–response flow, starting from the user request, checking the database, and returning either the schedule data or a “not found” response.

VIII. Create Personal Schedule Sequence Diagram

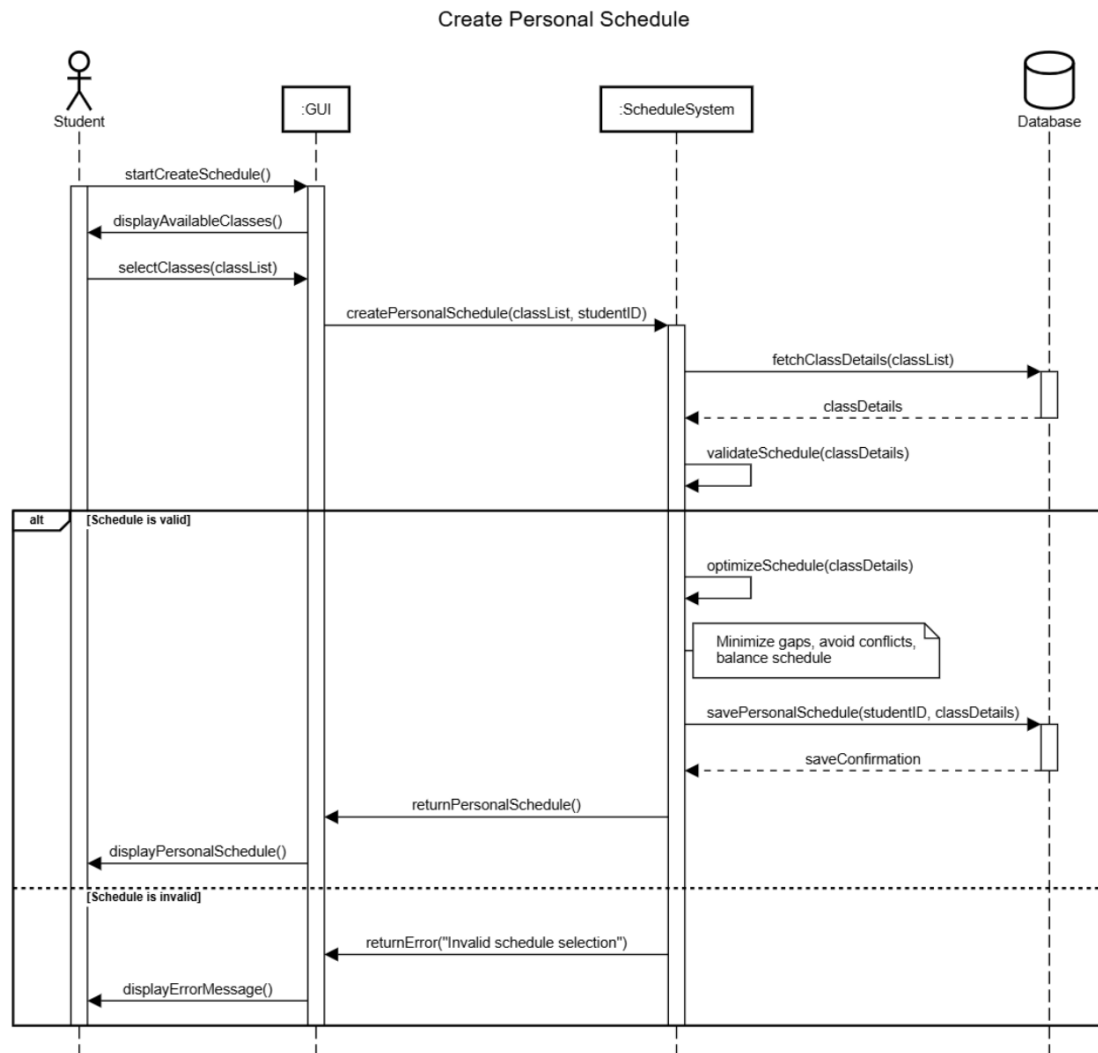


Figure 5.9: Create Personal Schedule Sequence Diagram

This diagram shows how a student builds a personal schedule by selecting available classes. The system checks for conflicts, validates the selection, and applies optimization rules before saving the final schedule. Objects involved:

- **Student (Actor):** Selects preferred classes to build a personal timetable.
- **GUI:** Displays available classes, receives selections, and shows results or errors.
- **ScheduleSystem (Controller):** Handles validation, optimization, and coordination with the database.
- **Database (Entity):** Provides class information and stores the final schedule.

All interactions are **synchronous**. The system follows a structured flow where class data is retrieved, validated for conflicts, optimized for better scheduling, and then saved to the database. Conditional logic is used to either return a completed schedule or an error message if validation fails.

5.3.2 ER Diagram

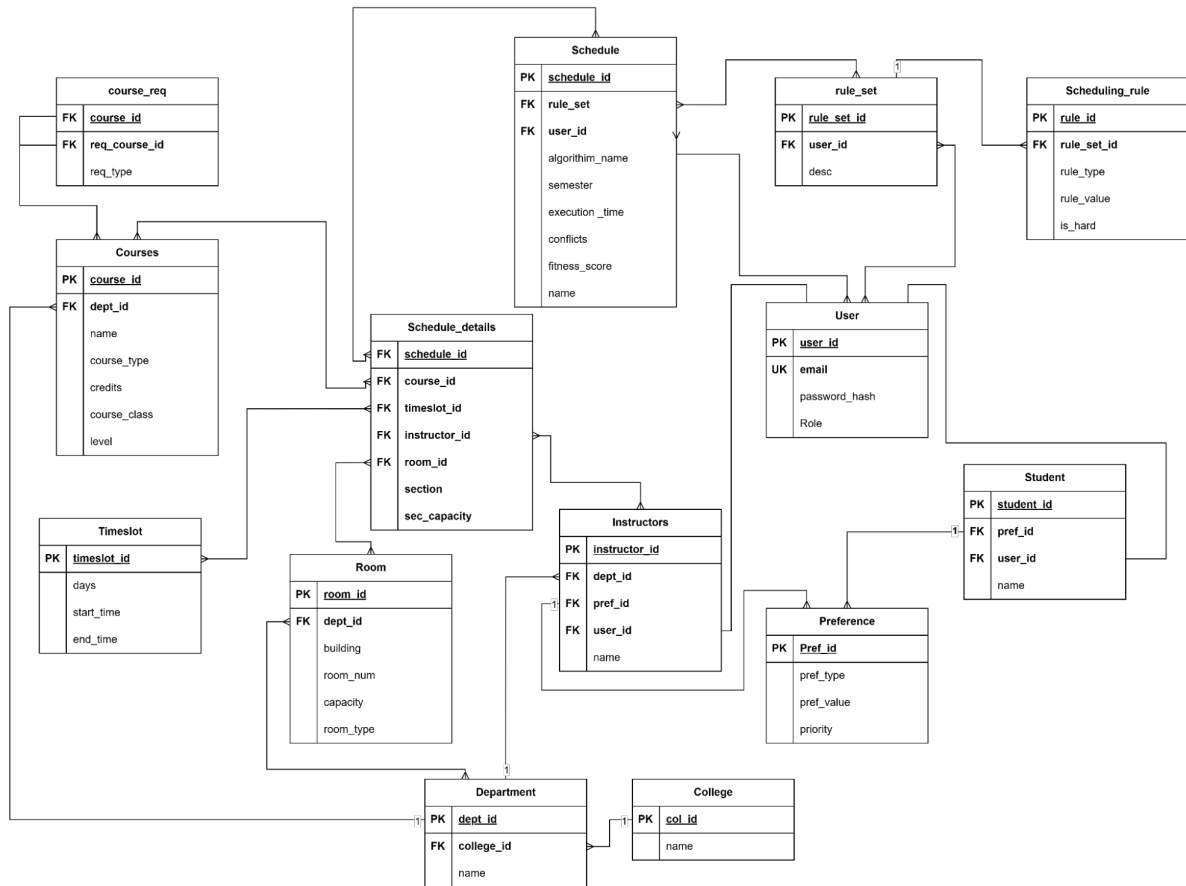


Figure 5.10: ER Diagram of the Database

The database consists of 14 tables, five of which are the primary tables used in the scheduling process: Courses, Timeslots, Rooms, and Instructors. The relationships between the main entities are designed to reflect the scheduling logic, where courses are assigned to instructors, rooms, and timeslots. These relationships ensure that constraints such as room capacity, instructor availability, and time conflicts are properly enforced during timetable generation.

5.4 User Interface Design (Prototype)

This stage of the prototype focuses on the admin-facing user interface.

Admin Dashboard Page: This page contains all the main functionalities of the system, including generating a schedule, managing data, viewing schedules, comparing algorithms, as well as an overview of the current system status and a section for recent schedules.

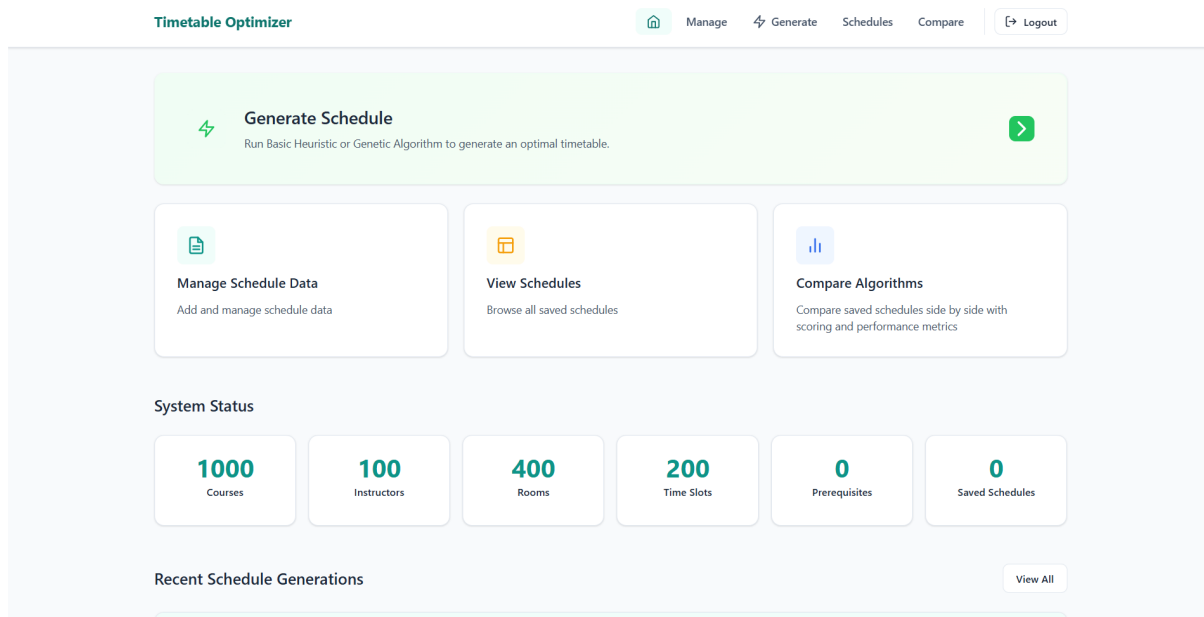


Figure 5.11: Admin UI Page

Manage Data Page: This page allows users to modify the data by adding or deleting. There are sections for courses, instructors, timeslots, and rooms. Also, from this page, preferences could be managed.

Manage Schedule Data

Add and manage all the data required for timetable optimization: courses, prerequisites, instructors, rooms, time slots, preferences, and constraints.

1. Courses

Course ID
e.g., CS101

Department ID
e.g., CS

Course Name
e.g., Intro to Computer Science

Course Type
-- Select Type --

Credits
e.g., 3

Course Class
e.g., Section A

Level
-- Select Level --

Expected Students
e.g., 40

Instructor
-- Select Instructor --

Add Course

COURSE ID	DEPT	NAME	TYPE	CREDITS	LEVEL	STUDENTS	INSTRUCTOR	ACTIONS
No courses added yet.								

Instructor Preferences

Preferred Teaching Time Start
08:00 AM

Preferred Teaching Time End
04:00 PM

Unavailable Times
e.g., Mon 12-14, Wed 8-10

Enter times when instructors are generally unavailable

Preferred Days
☐ Sun
 ☒ Mon
 ☒ Tue
 ☒ Wed
 ☒ Thu

☐ Limit max classes per day
 Maximum number of classes per instructor per day
 Max classes per day: 4

☐ Avoid back-to-back classes
 Give instructors breaks between consecutive classes

Save Preferences

Figure 5.12: Manage Data UI Page

Generate Schedule Page: This page contains components to run the algorithm, as well as provides a small description of the algorithms, so users with no background can benefit from it. When a schedule is generated, it appears in a tabular form with the ability to apply filters and only show courses scheduled in a building, timeslot, or from a department. Also, the result shows the metrics for evaluating the schedule.

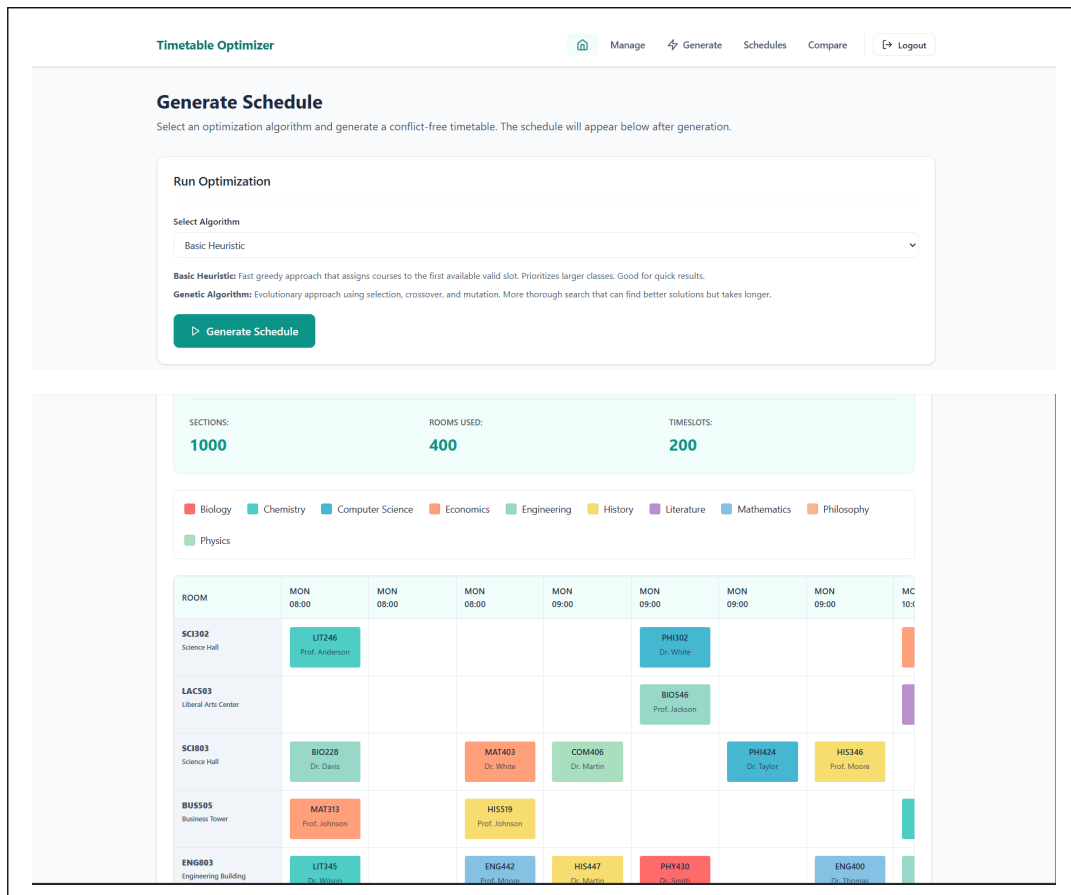


Figure 5.13: Generate Schedule UI Page

View Schedule Page: This page contains all saved and generated schedules.

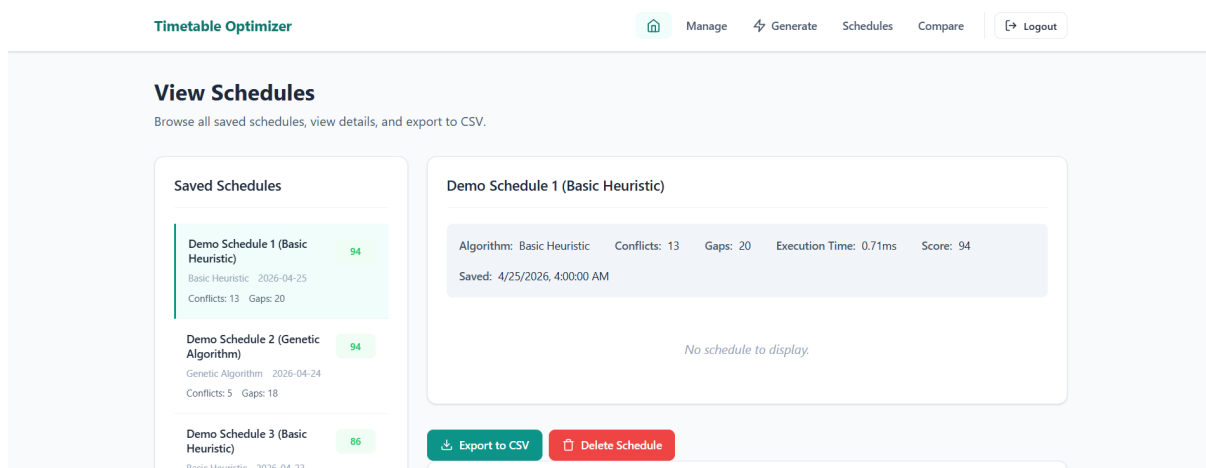


Figure 5.14: View Schedules UI Page

Compare Algorithm Page: This page contains all saved schedules in a list with their metrics. It also highlights the schedules with the best performance.

Timetable Optimizer

ManageGenerateSchedulesCompareLogout

Compare Algorithms

Compare saved schedules generated by different algorithms.

Saved Schedule Comparison

The fitness Score formula is calculated as : $100 - (\text{conflicts} \times 20) - (\text{gaps} \times 5)$. The row with the best score is highlighted.

SCHEDULE NAME	ALGORITHM	CONFLICTS	GAPS	EXECUTION TIME	FITNESS SCORE
Demo Schedule 1 (Basic Heuristic)	Basic Heuristic	13	20	0.71ms	94
Demo Schedule 2 (Genetic Algorithm)	Genetic Algorithm	5	18	2.50ms	94
Demo Schedule 3 (Basic Heuristic)	Basic Heuristic	12	19	4.37ms	86

Generate New ScheduleClear All Schedules

Figure 5.15: Compare UI Page

CHAPTER 6: Methodology and System Implementation

6.1 System Overview and Design Rationale

This work proposes a modular, web-based university timetabling system designed to support the development, integration, and evaluation of multiple optimization algorithms. The system is structured to separate data management, scheduling logic, constraint enforcement, and evaluation, enabling flexibility and extensibility in experimenting with different solution approaches.

A web-based architecture is adopted to facilitate centralized data handling and interactive visualization of generated timetables. The backend is designed as an algorithm-agnostic framework, allowing multiple scheduling strategies to be implemented and compared under a unified pipeline. The system operates in three main stages: (1) data acquisition and preprocessing, (2) schedule generation using a selected optimization algorithm, and (3) evaluation of the generated timetable using quantitative metrics. All source code of this project is stored and documented in this GitHub repository [28].

6.2 Data Modeling and Representation

Scheduling entities are represented using an object-oriented approach, where each entity is modeled as a structured class. The primary entities include:

- **Section:** Represents a scheduled class instance predefined as a triplet (`course_id`, `instructor_id`, `section_number`).
- **Room:** Contains capacity and type information.
- **TimeSlot:** Represents available scheduling periods.

This abstraction enables consistent data handling across different algorithms and simplifies the implementation of constraint checking.

6.3 Algorithm Management Framework

To support multiple optimization strategies, the system introduces an algorithm management framework based on a registry design pattern. This framework decouples algorithm selection from execution, enabling dynamic selection of scheduling methods at runtime. The scheduling engine acts as a controller that: (1) validates input data, (2) selects the appropriate algorithm, (3) executes the scheduling process, and (4) returns the generated timetable along with performance metrics.

6.4 Greedy Scheduling Algorithm

The greedy algorithm is implemented as a baseline heuristic due to its simplicity and computational efficiency [14, 13]. It constructs a timetable incrementally by assigning sections

one at a time based on priority. A conflict graph is first constructed, where each node represents a section and edges represent potential conflicts. Sections are then prioritized based on their conflict degree, ensuring that highly constrained sections are scheduled earlier.

For each section, the algorithm: (1) identifies all feasible room–timeslot combinations, (2) filters combinations that violate hard constraints, and (3) assigns the first valid combination based on predefined ordering. Hard constraints enforced during assignment include instructor availability, room availability, capacity constraints, campus compatibility, and room type requirements.

If no valid assignment is found, the section remains unscheduled. While this approach is computationally efficient, it may lead to suboptimal solutions due to its locally optimal decision-making process.

6.5 Genetic Algorithm for Timetabling

To address the limitations of the greedy approach, a genetic algorithm (GA) is implemented as a metaheuristic optimization technique capable of exploring a larger solution space [19, 20].

6.5.1 Solution Encoding

Each chromosome represents a complete timetable, where each gene corresponds to a scheduled section. A gene encodes the assignment of a section to a specific room and timeslot. If no valid assignment is found, the gene may contain null values, representing an unscheduled section.

6.5.2 Initialization

The initial population is generated using a randomized but constraint-aware procedure. For each section, a set of viable rooms and valid timeslots is precomputed. A feasible assignment is then selected while avoiding immediate conflicts such as instructor and room overlaps. If no fully valid assignment is available, a fallback random selection is used to maintain diversity in the population.

Listing 6.1: Population Initialization.

```

1 population = create_population(
2     size=10,
3     sections=sections,
4     rooms=rooms,
5     timeslots=timeslots
6 )

```

6.5.3 Genetic Operators

The GA employs the following operators:

Selection: A fitness-based selection mechanism is applied to retain high-quality solutions. The population is sorted in descending order based on fitness values, and only the top-performing 50% of individuals are selected for reproduction. This ensures that better solutions have a higher

probability of contributing to the next generation.

Crossover: A single-point crossover operator is used to combine two parent schedules. A random crossover point is selected, and the offspring is created by combining the first part of one parent with the second part of the other.

Listing 6.2: Single-Point Crossover

```
1 point = random.randint(1, len(parent1) - 1)
2 child = parent1[:point] + parent2[point:]
```

Mutation: The mutation operator introduces controlled randomness by modifying a randomly selected section within a schedule [21]. The implemented mutation is constraint-aware. When triggered, the operator applies one of two modifications with equal probability:

- **Room mutation:** A new viable room is selected from the precomputed set of capacity- and type-compatible rooms. The selected room must not already be occupied at the section's current timeslot.
- **Timeslot mutation:** A new timeslot is selected satisfying two conditions simultaneously: the section's instructor must be free at that timeslot, and the section's currently assigned room must also be free.

6.5.4 Hyperparameters

Table 6.1: GA Hyperparameters

Hyperparameter	Symbol	Value
Population size	N_{pop}	10
Number of generations	G	20
Crossover rate	p_c	0.85
Mutation rate	p_m	0.03
Selection ratio (truncation)	σ	0.50
Number of independent runs	N_{runs}	30

The hyperparameters were selected based on established practice in GA literature [22]. The crossover rate ($p_c = 0.85$) and mutation rate ($p_m = 0.03$) follow standard recommendations for binary-encoded GAs. The values of population size ($N_{\text{pop}} = 10$) and number of generations ($G = 20$) were chosen conservatively to maintain tractable execution times on the available hardware, at the cost of potentially limiting the solution quality achievable by the algorithm.

6.5.5 GA Workflow

1. Initialize population P with random feasible schedules.
2. For generation = 1 to G :

- a. Evaluate fitness of each schedule in P .
 - b. Select top-performing individuals (selection).
 - c. Generate new population: select two parents, apply crossover, apply mutation, add offspring to new population.
 - d. Replace P with new population.
3. Return best schedule from final population.

6.6 Constraint Management

A centralized constraint management module ensures consistency across all algorithms. This module defines and enforces hard constraints including: instructor conflict constraints, room conflict constraints, capacity constraints, room type compatibility, and campus restrictions. By centralizing constraint logic, the system ensures that all algorithms operate under the same feasibility conditions, enabling fair comparison of results.

6.7 Evaluation and Fitness Function

The quality of each generated schedule is assessed using a normalized, penalty-based fitness function. The function takes two inputs: the number of unscheduled sections and the total number of constraint violations, both normalized relative to the total number of sections to ensure comparability across different dataset sizes. Two penalty terms are computed separately:

$$\text{unscheduled_penalty} = \frac{\text{unscheduled}}{\text{total_sections}} \times 2.0$$

$$\text{conflict_penalty} = \frac{\text{conflicts}}{\text{total_sections}} \times 1.0$$

The weight of 2.0 assigned to unscheduled sections reflects the design priority that a missing class is a more severe outcome than a conflicted one — a conflicted section at least exists in the schedule and may be manually correctable, whereas an unscheduled section represents a complete failure to place a course. This weighting scheme follows the constraint hierarchy approach described by [23] and the weighted penalty normalization framework of [24]. The total penalty is the sum of both terms, and the final fitness score is computed as:

$$\text{Fitness} = \frac{1}{1 + \text{total_penalty}}$$

This formulation maps all fitness values to the range (0, 1], where a score of 1.0 represents a fully scheduled, conflict-free timetable. Higher fitness values consistently indicate better solutions. This implementation operationalizes the theoretical objective function defined in Chapter 3. The unscheduled penalty directly corresponds to the $\cdot\Phi_{\text{wait}}$, and the conflict penalty corresponds to $\cdot\Phi_{\text{conf}}$. The components $\cdot\Phi_{\text{util}}$ and $\cdot\Phi_{\text{spread}}$ from the theoretical model are enforced implicitly through the constraint management module during viable room selection and assign-

ment rather than through the fitness score, which focuses computational pressure on the two most critical scheduling outcomes

6.8 Database Integration

The system is integrated with a relational database implemented using Supabase PostgreSQL. The database stores all scheduling data, including sections, rooms, time slots, and constraints. This integration enables the system to operate on real-world datasets rather than synthetic inputs, improving the validity of experimental results. Additionally, generated schedules are stored for later retrieval and comparison.

6.9 Implementation Technologies

The system is implemented using a combination of modern web and backend technologies:

- **Frontend:** TypeScript, HTML, CSS, JavaScript
- **Frameworks:** React, Next.js, Streamlit (for rapid visualization)
- **Backend:** Python
- **Database:** Supabase PostgreSQL
- **Tools:** Git, GitHub, V0 AI (for UI elements generation)
- **Deployment:** Vercel

CHAPTER 7: Verification, Validation, and Evaluation

7.1 Evaluation Methodology

The evaluation covers two distinct dimensions. First, functional correctness is verified through structured testing of individual components and integrated workflows. Second, optimization performance is evaluated quantitatively by comparing the greedy and genetic algorithms across five metrics: scheduling rate, conflict rate, fitness score, execution time, and algorithm stability. The dataset used throughout the evaluation consists of 4,698 sections, 375 rooms, and 479 timeslots drawn from real university scheduling data.

7.2 Test Strategy

The system will be tested using several testing approaches to ensure its correctness and reliability. Unit testing will be applied to verify individual components such as the login module, data management, and timetable generation functions. Integration testing will be conducted to ensure that different parts of the system, including the optimization algorithm and the database, work together correctly. System testing will evaluate the complete functionality of the application to confirm that it generates valid and conflict-free timetables. In addition, user acceptance testing will be performed to ensure that the system meets user requirements and is easy to use.

7.2.1 Unit Testing

The fitness function was unit tested by constructing synthetic schedules with known properties.

Table 7.1: Unit Test Cases – Fitness Function

TC ID	Description	Input Values	Expected Outcome	Actual Output	Pass/Fail
TC1	Fully scheduled, conflict-free schedule	10 sections, 0 unscheduled, 0 conflicts	Fitness = 1.0	Fitness = 1.0	Pass
TC2	Half unscheduled schedule	10 sections, 5 unscheduled, 0 conflicts	Fitness = 0.5	Fitness = 0.5	Pass
TC3	Fully scheduled with maximum conflicts	10 sections, 0 unscheduled, 10 conflicts	Fitness = 0.5	Fitness = 0.5	Pass
TC4	Instructor conflict detection	Same instructor assigned to two sections at same timeslot	Conflict detected	Conflict detected	Pass
TC5	Room conflict detection	Same room assigned to two sections at same timeslot	Conflict detected	Conflict detected	Pass
TC6	Valid assignment	Different instructors, rooms, and timeslots	No conflict	No conflict	Pass

7.2.2 Integration Testing

The integration between the scheduling engine and the constraint module was verified by running the greedy algorithm on a reduced dataset of 50 sections and manually inspecting the output for constraint violations. All 50 sections were assigned without instructor or room conflicts, confirming that constraint checking is correctly applied during assignment. Database integration was verified by confirming that generated schedules persisted correctly and could be retrieved without data loss across sessions

Table 7.2: Integration Test Cases

TC ID	Description	Input Values	Expected Outcome	Actual Output	Pass/Fail
TC7	Scheduling engine + constraint module	50 sections dataset	All sections scheduled with no conflicts	All sections scheduled, no conflicts	Pass
TC8	Backend + database persistence	Generated schedule saved to database	Data stored and retrieved correctly	Data retrieved correctly	Pass

7.2.3 System Testing

Table 7.3: System Test Cases

TC ID	Description	Input	Expected Outcome	Actual Output	Pass/Fail
TC9	Full dataset – Greedy algorithm	4698 sections	High scheduling rate, no conflicts	92.7% scheduled, 0 conflicts	Pass
TC10	Full dataset – Genetic algorithm	4698 sections	High scheduling rate, minimal conflicts	96% scheduled, 196 conflicts	Pass

7.2.4 User Acceptance Testing

Table 7.4: User Acceptance Test Cases

TC ID	Description	Input	Expected Outcome	Actual Output	Pass/Fail
TC11	Dashboard usability	User navigates dashboard	Easy navigation	Navigation successful	Pass
TC12	Schedule filtering	Filter by course/room/-time	Correct filtered results	Correct results displayed	Pass
TC13	Data management	Add/edit/delete schedule data	Changes reflected correctly	Changes applied correctly	Pass

7.3 Optimization Evaluation Metrics

To evaluate the quality of generated timetables, the following quantitative metrics are used:

- **Number of Scheduled Sections:** Measures how many sections are successfully assigned to valid room–timeslot pairs.
- **Number of Unscheduled Sections:** Indicates incomplete scheduling and is heavily penalized.
- **Constraint Violations:** Includes instructor conflicts, room conflicts, and capacity violations.
- **Fitness Score:** A normalized metric combining constraint violations and unscheduled sections into a single value between 0 and 1.
- **Execution Time:** Measures the computational efficiency of each algorithm.

7.4 Experimental Results Overview

The genetic algorithm is executed multiple times to account for its stochastic nature. Statistical measures computed include: best fitness score, worst fitness score, average fitness score, and standard deviation.

7.5 Security and Privacy Considerations

Table 7.5: Identified Risks and Mitigation Strategies

Risk	Description	Impact	Mitigation	How Addressed
Weak Passwords	Users choose insecure passwords	High	Enforce strong password policies	Supabase Auth enforces minimum password length and complexity requirements
SQL Injection	Malicious input targeting database queries	High	Input validation and parameterized queries	All database interactions use Supabase's parameterized query API; raw SQL strings are not constructed from user input
Unauthorized Access	Unauthorized users accessing the system	High	Authentication and access control	Role-based access implemented via Supabase Auth
Data Leakage	Exposure of sensitive data	Medium	Secure storage and restricted access	Database access is restricted to authenticated sessions; no sensitive data is exposed in API responses to unauthorized roles

7.6 Functional Requirement Verification

Table 7.6: Requirements Test Cases

TC ID	Test Method	Requirement Description	Req ID	Status
TC-01	Manual system test	Allow admin users to define courses, instructors, rooms, time slots, and constraints	FR-01	Pass
TC-02	Full dataset run, constraint validation	Generate feasible timetables satisfying hard constraints	FR-02	Pass
TC-03	Greedy and GA both executed successfully	Support execution of multiple scheduling algorithms for comparison	FR-03	Pass
TC-04	Manual UI test of edit functionality	Allow admins to modify generated timetables	FR-04	Pass
TC-05	Unit test with synthetic conflicting data	Detect and prevent instructor and room conflicts	FR-05	Pass
TC-06	Metrics verified against manually calculated values	Compute evaluation metrics (conflicts, gaps, execution time)	FR-06	Pass
TC-07	Visual inspection of UI output	Display timetables in a clear tabular format	FR-07	Pass
TC-08	Manual test with invalid inputs	Validate user input and provide feedback if generation fails	FR-08	Pass

7.7 Software Quality Assurance

Table 7.7: Software Quality Assessment

Quality Attribute	Evaluation Method	Evidence	Assessment
Efficiency	Execution time measurement	Greedy: 1.33 s; GA: ≈ 120 s per run	Meets acceptance criteria. The greedy approach is significantly faster; the GA trades performance for optimization capability.
Reliability	30-run GA stability analysis	Standard deviation $\sigma = 0.0002$ across 30 runs	High reliability. The GA demonstrates consistent convergence with minimal variation between runs.
Correctness	Constraint validation on full dataset	Greedy: 0 violations; GA: 196 room conflicts	Greedy approach is fully correct. GA is near-correct but requires further refinement.
Usability	Team walkthrough testing	All workflows completed without external assistance	Adequate for prototype stage.
Maintainability	Architectural review	Modular design, use of registry pattern, version control integration	Well-supported. The system structure enables easy updates, debugging, and future extensions.
Scalability	Single dataset evaluation	Tested on 4,698 sections using real university data	Demonstrated capability at realistic scale. However, multi-scale and stress testing are still needed.

CHAPTER 8: Results and Discussion

The proposed timetabling system is evaluated using a real-world university dataset consisting of 4,698 sections, 375 rooms, and 479 time slots. The dataset includes a manually constructed base schedule derived from historical scheduling practices. To assess the effectiveness of the proposed approach, the base schedule is compared against schedules generated using the greedy and genetic algorithms under a unified constraint validation and fitness evaluation framework.

I. Baseline Schedule Analysis

The base schedule, although complete (100% scheduling rate), exhibits a high number of constraint violations when evaluated using the system's constraint model.

Table 8.1: Conflict Breakdown (Base Schedule)

Instructor	Room	Campus	Room type	Department	Timeslot	Capacity
558	244	950	338	820	901	301

The high conflict count in the base schedule (4,112) does not imply that the manually constructed schedule is of poor quality in practice. Rather, it reveals a fundamental challenge in computational timetabling: real-world schedules frequently incorporate implicit institutional knowledge, historical exceptions, and undocumented agreements that cannot be fully captured in a formal constraint model [5].

II. Greedy Schedule

The greedy algorithm produces a fully conflict-free schedule in 1.33 seconds, demonstrating strong feasibility under the defined constraint model. However, this comes at the cost of completeness: 344 sections remain unscheduled, yielding a scheduling rate of 92.7%.

III. Genetic Schedule

The genetic algorithm achieves a better balance between feasibility and completeness. Total conflicts: 196; unscheduled sections: 149; scheduling rate: 96%. The GA solution contains 196 residual room conflicts and zero instructor conflicts.

Table 8.2: Conflict Breakdown (Genetic Algorithm)

Instructor	Room	Campus	Room type	Department	Timeslot	Capacity
0	196	0	0	0	0	0

8.1 Comparison and Evaluation

Table 8.3: Constraint Satisfaction Metrics

Metric	Base	Greedy	Genetic
Total sections	4698	4698	4698
Scheduled sections	4698	4354	4549
Unscheduled sections	0	344	149
Scheduling rate (%)	100%	92.7%	96%
Conflict count	4112	0	196
Conflict rate (%)	87.5%	0%	4.17%

Table 8.4: Computational Efficiency

Metric	Greedy	Genetic (per run)
Execution time	1.33 s	120 s
Sections per second	3,274	1.26

Table 8.5: Solution Quality Score

Metric	Base	Greedy	Genetic
Fitness score	0.533	0.889	0.904
Rank	3	2	1

8.2 Genetic Algorithm Stability

To evaluate the consistency of the genetic algorithm, the optimization process was executed 30 times independently:

- Best fitness: 0.9049
- Worst fitness: 0.9042
- Average fitness: 0.9044
- Standard deviation: 0.0002

The very low standard deviation indicates that the GA produces highly stable results across runs, suggesting strong convergence behavior despite its stochastic nature.

8.3 Discussion

The results demonstrate a clear trade-off between feasibility, completeness, and computational cost:

- The base schedule achieves full coverage but violates many formal constraints.
- The greedy algorithm ensures complete feasibility but sacrifices coverage.
- The genetic algorithm provides the best balance, achieving high coverage with minimal conflicts.

A key observation is that constraint modeling significantly impacts solution quality. As more constraints were explicitly defined and enforced, both algorithms produced improved results. However, it is not possible to claim optimality due to the complexity of the problem, incomplete modeling of real-world constraints, and the presence of dataset-specific exceptions.

8.4 Limitations

The evaluation is subject to several limitations that impact its accuracy and comprehensiveness. Not all real-world constraints are formally captured, which can lead to a lack of consideration for certain critical factors. Additionally, some exceptions specific to the dataset are not modeled. The genetic algorithm employed has a relatively high runtime, which may hinder the efficiency of the evaluation process. Furthermore, the definitions of constraints are only partially tailored to the dataset, leaving room for discrepancies in the evaluation outcomes.

CHAPTER 9: Conclusion and Future Direction

9.1 Conclusion

This project tackled the university timetabling problem by creating a modular, web-based optimization system that generates and evaluates conflict-aware academic schedules. The system operates on the principle that no single scheduling algorithm is universally superior, prompting the development of two distinct algorithms: a greedy constructive heuristic and a genetic algorithm, evaluated under a unified constraint validation framework.

Using real data from the University of Sharjah, the study compared three types of schedules: a manually created base schedule, the greedy-generated schedule, and the GA-generated schedule. The base schedule achieved 100% section coverage but had a 87.5% conflict rate, revealing that manual schedules incorporate implicit knowledge and historical exceptions that are challenging to formalize.

The greedy algorithm produced a conflict-free schedule in 1.33 seconds but left 344 sections unscheduled. In contrast, the genetic algorithm allowed some conflicts (4.17% rate) but scheduled 4,549 sections, reducing the number of unscheduled sections by 57%. This approach demonstrated a fitness score of 0.904, with strong stability across multiple runs.

Improvements in constraint definitions during development significantly enhanced solution quality, indicating that effective constraint modeling is as critical as algorithm choice in timetabling optimization.

9.2 Future Work

- **Frontend and Backend Integration:** The most immediate priority is completing the integration between the frontend and the Python backend. Full integration will enable real-time schedule generation, live constraint feedback, and an administrator-facing interface that reflects the complete system design.
- **Hyperparameter Tuning and Algorithm Efficiency:** A systematic tuning study varying population size, generation count, crossover rate, and mutation rate across multiple dataset configurations would likely yield measurable improvements in both solution quality and execution time.
- **Additional Optimization Algorithms:** Candidate algorithms include simulated annealing, hybrid methods combining constructive heuristics with local search refinement, particle swarm optimization, and Tabu search.
- **Instructor Preference and Workload Modeling:** A future phase should model instructor availability, preferred teaching times, and maximum weekly load as explicit constraints and optimization objectives.

- **Student-Centered Scheduling:** Extending the system to support student-facing schedule creation, where students select courses and the system generates conflict-free personal timetables, would significantly broaden the system's practical utility.

Bibliography

- [1] Tassopoulos, I. X., Iliopoulou, C. A., Katsaragakis, I. V., & Beligiannis, G. N. (2023). An effective local particle swarm optimization-based algorithm for solving the school timetabling problem. *Algorithms*, 16(6), 291.
- [2] Austero, L. D., Sison, A. M., Matias, J. B., & Medina, R. P. (2022). Solving course timetabling problem using whale optimization algorithm. In *Proceedings of the 8th International Conference on Information Technology Trends (ITT)* (pp. 160–166).
- [3] Hossin, M. B., Rushdi, A. M., & Tuah, N. M. (2025). Pheromone-guided university timetable construction using ant colony optimization: A case study. In *Proceedings of the 14th International Conference on Information Technology in Asia (CITA)* (pp. 108–113).
- [4] Chu, S.-C., Chen, Y.-T., & Ho, J.-H. (2006). Timetable scheduling using particle swarm optimization. In *Proceedings of the International Conference on Innovative Computing, Information and Control (ICICIC)*.
- [5] Bashab, A., et al. (2023). Optimization techniques in university timetabling problem: Constraints, methodologies, benchmarks, and open issues. *Computers, Materials & Continua*, 74(3), 6461–6484.
- [6] Qu, R., Burke, E. K., McCollum, B., Merlot, L. T. G., & Lee, S. Y. (2009). A survey of search methodologies and automated system development for examination timetabling. *Journal of Scheduling*, 12(1), 55–89.
- [7] Socha, K., & Knowles, J. (2007). A comparison of various heuristics applied to the university timetabling problem. *Journal of Heuristics*, 13(1), 87–115.
- [8] Rodríguez-Maya, N., Flores, J. J., & Rodríguez-Rangel, H. (2016). Performance comparison of evolutionary algorithms for university course timetabling problem. *Computación y Sistemas*, 20(4), 623–637.
- [9] Shaker, K., Abdullah, S., & Alqudsi, A. (2012). A differential evolution algorithm for the university course timetabling problem. In *Proceedings of the 4th International Conference on Data Mining and Optimization (DMO)*.
- [10] Kakkar, M. K., et al. (2021). Class schedule generation using evolutionary algorithms. *Journal of Physics: Conference Series*, 1950(1).
- [11] George, T. B., & Chung, J. (2000). Applying evolution strategies to a university timetabling system. Retrieved from <https://www.researchgate.net/publication/220739805>.
- [12] Mazlan, M., Makhtar, M., Khairi, A. F. K. A., & Mohamed, M. A. (2019). University course timetabling model using ant colony optimization algorithm approach. *Indonesian*

- Journal of Electrical Engineering and Computer Science*, 13(1), 72–76.
- [13] Hassan-tech-pro. (2025). GitHub - Hassan-tech-pro/Schedulr. <https://github.com/Hassan-tech-pro/Schedulr>.
- [14] Hussin, B., Samad, A., Basari, H., Shibghatullah, A., & Asmai, S. (2011). Exam timetabling using graph colouring approach.
- [15] UniTime. (2019). UniTime | University Timetabling. <https://www.unitime.org>.
- [16] aSc Timetables. <https://www.asctimetables.com>.
- [17] FET - Free Timetabling Software. <https://lalescu.ro/liviu/fet/>.
- [18] Scientia Ltd. (2026). Scientia Syllabus Plus. <https://scientia-syllabus-plus.software.informer.com/>.
- [19] FiziQaiser. (2025). GitHub - FiziQaiser/Genetic-Algorithm-Timetable-Scheduling. <https://github.com/FiziQaiser/Genetic-Algorithm-Timetable-Scheduling>.
- [20] Vijini Mallawaarachchi. (2020). Using Genetic Algorithms to Schedule Timetables. *Medium; TDS Archive*.
- [21] Deb, K. (2000). An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2–4), 311–338.
- [22] Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley Professional.
- [23] Ceschia, S., Di Gaspero, L., & Schaerf, A. (2022). Educational Timetabling: Problems, Benchmarks, and State-of-the-Art Results. *European Journal of Operational Research*, 308(1).
- [24] Burke, E. K., Elliman, D. G., & Weare, R. (1994). A University Timetabling System Based on Graph Colouring and Constraint Manipulation. *Journal of Research on Computing in Education*, 27(1), 1–18.
- [25] Irene, Deris, S., & Mohd, Z. (2009). University course timetable planning using hybrid particle swarm optimization. *Genetic and Evolutionary Computation Conference*.
- [26] Badoni, R. P., & Gupta, D. K. (2015). A hybrid algorithm for university course timetabling problem. *Innovative Systems Design and Engineering*, 6(2), 60–66.
- [27] Banda, S., & Halubanza, B. (2025). A Hybrid Genetic Algorithm–Tabu Search Approach for AI-Driven Exam Timetabling in Higher Education. *Proceedings of International Conference for ICT (ICICT) - Zambia*, 7(1), 298–306.
- [28] Github repository: Scheduling-optimization-tool. (2026). GitHub. <https://github.com/ftmmz/scheduling-optimization-tool>.